

ПРОФИЛИРАНА МАТЕМАТИЧЕСКА ГИМНАЗИЯ
„КОНСТАНТИН ВЕЛИЧКОВ“ ГР.
ПАЗАРДЖИК

Д И П Л О М Н А Р А Б О Т А

Тема: Разработка на уеб приложение „ЛапиХъб“
Дипломен проект за придобиване на трета степен на професионална квалификация – част от теорията на професията

Дипломант: Мартин Красимиров Найденов
Специалност: „Приложно програмиране“ код 4810301
Професия: „Приложен програмист“ код 481030
Консултант: Валентина Малинова

Пазарджик
2026

Съдържание

СЪДЪРЖАНИЕ ..... **Грешка! Показалецът не е дефиниран.**

Глава 1. УВОД 6

Глава 2. Теоретични основи на електронната търговия 8

2.1. Същност и видове електронна търговия 8

2.2. Архитектурният модел Model–View–Controller 9

2.3. ASP.NET Core MVC като платформа за уеб разработка 10

2.4. Entity Framework Core и подходът Code First 11

2.5. SQL Server като релационна база данни 12

2.6. ASP.NET Core Identity – потребители, роли и сигурност 12

2.7. Потребителски интерфейс, Razor Views и Bootstrap 5 13

Глава 3. Анализ на изискванията към системата „ЛапиХъб“ 14

3.1. Предназначение на системата и целева аудитория 14

3.2. Потребителски роли и нива на достъп 15

3.3. Функционални изисквания към публичната част 16

3.4. Функционални изисквания към потребителската част 17

3.5. Функционални изисквания към административната част 18

3.6. Нефункционални изисквания 18

3.7. Основни сценарии на използване 19

Глава 4. Проектиране и реализация на системата „ЛапиХъб“ 20

4.1. Обща структура на проекта 20

4.2. Конфигурация на приложението и ролята на Program.cs 21

4.3. ApplicationDbContext и конфигурация на данните 23

4.4. Потребителският модел ApplicationUser 24

4.5. Модел Category 26

4.6. Модел Product 27

4.7. Модел CartItem 29

4.8. Модел Order 30
4.9. Модел OrderItem 31
4.10. Контролери и основна бизнес логика 32
4.10.1. HomeController 33
4.10.2. ProductsController 34
4.10.3. DashboardController 35
4.10.4. CartController 36
4.10.5. OrdersController 37
4.10.6. AdminController 38
4.10.7. AdminCategoriesController 39
4.10.8. AdminProductsController 40
4.10.9. AdminOrdersController 41
4.10.10. AdminUsersController 42
4.11. Изгледи и потребителски интерфейс 43
4.12. Валидация на входните данни 45
4.13. Роли, авторизация и сигурност 46
4.14. Начални данни и seed логика 47
4.15. Обобщение на основния функционален поток 47
Глава 5. Тестване на системата 49
5.1. Цели на тестването 49
5.2. Тестова среда 49
5.3. Тестване на публичната част 50
5.4. Тестване на регистрация, вход и роли 51
5.5. Тестване на количка и checkout 51
5.6. Тестване на потребителските поръчки 52
5.7. Тестване на административната част 52
5.8. Обобщение на резултатите от тестването 53

ЗАКЛЮЧЕНИЕ 55
ИЗПОЛЗВАНА ЛИТЕРАТУРА 56
Ръководство за потребителя 57
Работа със системата като гост 57
Регистрация и вход 58
Работа с таблото на потребителя 59
Работа с количката 59
Създаване на поръчка 60
Преглед на поръчките 61
Работа със системата като администратор 62
Практически насоки за работа 62
Инсталация, пренасяне и поддръжка на проекта 64
Необходими условия за стартиране 64
Създаване на базата данни 64
Seed на роли и администратор 65
Често срещани проблеми и решения 65
Поддръжка и възможности за бъдещо разширяване 65
ПРИЛОЖЕНИЯ 67
Приложение 1 67
Приложение 2. 68
Приложение 3: Архитектура на приложението 69
Приложение 4: ApplicationUser 70
Приложение 5: CartItem.cs 70
Приложение 6: Category.cs 71
Приложение 7: Order.cs 72
Приложение 8: OrderItem.cs 72
Приложение 9: OrderStatus.cs 73

Приложение 10: PaymentMethod.cs 73
Приложение 11: Product.cs 74
Приложение 12: _Layout.cshtml 74
Приложение 13: DbSeeder.cs 76
Приложение 14: Program.cs 76
Приложение 15: ApplicatiomDbContext.cs 77
Приложение 16: UML Диаграма 78

Глава 1. УВОД

Настоящата дипломна документация представя проектирането и реализацията на уеб приложение с наименование „ЛапиХъб“, предназначено за електронна търговия с продукти за домашни любимци. Проектът е реализиран като онлайн магазин, чрез който потребителите могат да разглеждат каталог от продукти, да търсят и филтрират артикули, да добавят избрани стоки в количка, да създават поръчки и да следят собствената си история на покупки. Отделно от потребителската част системата включва и административен панел, чрез който се извършва управление на каталога, категориите, потребителите и поръчките.

Темата на разработката е избрана с оглед на високата практическа стойност на електронната търговия и нарастващата нужда от уеб базирани системи, които да предоставят удобен и подреден интерфейс за продажба на специализирани продукти. Пазарът на стоки за домашни любимци е добър пример за такава специализация, тъй като предлага разнообразни категории – храни, лакомства, играчки, аксесоари, продукти за хигиена и грижа – които могат да бъдат организирани в каталожна структура и управлявани изцяло чрез уеб приложение.

В разработката са използвани съвременни технологии от екосистемата на Microsoft. Основата е .NET 8 и ASP.NET Core MVC, което осигурява ясна архитектура по модела Model–View–Controller. За достъп до данните и работа с релационната база данни се използва Entity Framework Core чрез подхода Code First, а за надеждно съхранение на информацията е избран SQL Server. Управлението на потребители, вход, регистрация и роли е реализирано чрез ASP.NET Core Identity. Потребителският интерфейс е създаден с Razor Views и Bootstrap 5.

Целта на проекта е да бъде разработена завършена, функционална и поддържаема електронна система, която да моделира реалистичен сценарий на онлайн магазин за продукти за домашни любимци. От една страна системата трябва да бъде удобна за крайния потребител, а от друга – да предоставя надежден административен инструмент за управление на съдържанието. Това означава не само визуализиране на продукти, но и реализиране на пълния жизнен цикъл на онлайн покупката – от разглеждане на каталога до създаване на поръчка и проследяване на нейните детайли.

При разработването на „ЛапиХъб“ е поставен акцент върху добрата организация на кода, разделянето на отговорностите между отделните слоеве и

създаването на сигурна среда за работа с различни типове потребители. В системата са дефинирани три логически роли – гост, регистриран потребител и администратор – като всяка от тях разполага с различен набор от права. По този начин приложението покрива както публичната част на един онлайн магазин, така и защитената потребителска и административна област.

Настоящата документация проследява целия процес по създаването на приложението. В теоретичната част се разглеждат основните понятия, свързани с електронната търговия и използваните технологии. След това се прави анализ на изискванията към системата и се представят нейните основни функционални и нефункционални характеристики. Най-голямо внимание е отделено на проектирането и реализацията на приложението, включително структурата на проекта, базата данни, моделите, контролерите, изгледите, сигурността и тестовите сценарии.

Документацията има две основни цели. Първата е да представи разработката като завършен дипломен проект, който демонстрира усвоени знания и практически умения в областта на уеб разработката. Втората е да служи като техническо и потребителско описание на системата, което може да бъде използвано както за оценяване, така и за бъдещо поддържане, разширяване и адаптиране на приложението.

Създаването на проект като „ЛапиХъб“ показва как чрез съчетаването на добра архитектура, правилно моделиране на данни, сигурна идентификация на потребителите и добре организиран интерфейс може да бъде изградена реалистична e-commerce система. Макар проектът да е учебен по своята цел, неговата логика и технологична основа са напълно приложими и в реална работна среда.

Глава 2. Теоретични основи на електронната търговия

Електронната търговия представлява съвкупност от процеси, технологии и организационни модели, чрез които покупко-продажбата на стоки и услуги се извършва в дигитална среда. За разлика от традиционната търговия, при която физическото присъствие в обект е задължително, при електронната търговия комуникацията между търговец и клиент се осъществява чрез уеб сайт или друго онлайн приложение. Това позволява по-бърз достъп до информация, улеснено сравнение между продукти и автоматизирано обработване на заявки.

Една съвременна e-commerce система обикновено включва няколко основни модула: каталог от продукти, категории, търсене, филтри, количка, поръчки, управление на потребители и административен панел. Именно комбинацията от тези модули превръща един обикновен продуктов сайт в реална платформа за търговия. В разглеждания проект „ЛапиХъб“ всички тези елементи са реализирани като взаимно свързани функционалности, изградени в рамките на единно ASP.NET Core MVC приложение.

За да бъде една такава система надеждна, тя трябва да стъпи върху ясна архитектура и върху технологии, които позволяват правилно моделиране на данни, сигурна идентификация на потребителите и лесно разширяване. Именно затова в тази глава се разглеждат теоретичните понятия, които стоят в основата на проекта: същност на онлайн търговията, MVC архитектура, ASP.NET Core MVC, Entity Framework Core, SQL Server, ASP.NET Identity и ролята на потребителския интерфейс.

2.1. Същност и видове електронна търговия

Електронната търговия може да се дефинира като извършване на търговски операции чрез електронни комуникационни средства, най-често интернет. Тя включва представяне на продукти, приемане на заявки, комуникация с клиента, обработка на плащания, проследяване на поръчки и поддръжка след продажбата. В най-широк смисъл електронната търговия е дигитална форма на организация на търговски процеси, при която информационният поток и част от бизнес логиката се автоматизират.

В практиката съществуват различни видове електронна търговия. Най-разпространен е моделът business-to-consumer (B2C), при който фирма продава

директно на краен клиент. Точно към този тип принадлежи и проектът „ЛапиХъб“, защото магазинът предлага продукти за домашни любимци на индивидуални потребители. Съществуват и други модели, например business-to-business (B2B), consumer-to-consumer (C2C) и consumer-to-business (C2B), но те имат различна организация на процесите и различни изисквания към интерфейса и данните.

Особено важна особеност на електронната търговия е, че клиентът взема решение за покупка без физически контакт с продукта. Поради това уеб приложението трябва да предоставя ясна, подробна и добре структурирана информация. В случая с „ЛапиХъб“ това означава продуктите да бъдат организирани по категории, да имат снимки, цена, марка и описание, а каталогът да позволява търсене, филтриране и сортиране. По този начин системата компенсира липсата на физически контакт чрез по-добро информационно представяне.

Друга особеност на e-commerce системите е, че те трябва да осигурят едновременно удобство и контрол. За крайния потребител това се изразява в лесна навигация, интуитивна количка и кратък checkout процес. За администратора това означава наличие на панел за управление на данните и механизмите, чрез които магазинът може да бъде поддържан и развиван. Следователно електронната търговия е не само потребителски интерфейс, но и организационна система за управление на информация и търговски операции.

2.2. Архитектурният модел Model–View–Controller

MVC е един от най-широко използваните архитектурни модели при разработването на уеб приложения. Неговата основна идея е да раздели системата на три логически части – Model, View и Controller – така че всяка от тях да има ясно дефинирана отговорност. Това намалява сложността, прави кода по-четим и улеснява поддръжката на по-големи приложения.

Компонентът Model описва данните и бизнес обектите на приложението. В контекста на „ЛапиХъб“ моделите включват класове като Product, Category, CartItem, Order, OrderItem и ApplicationUser. Те представят същностите в системата, техните свойства, релации и правила за валидиране. По този начин моделът служи като логическа основа на приложението.

Компонентът View отговаря за визуалното представяне на данните. Това са страниците, които потребителят вижда в браузъра – начална страница, каталог, детайли за продукт, количка, табло, поръчки и административни екрани. В проекта

изгледите са реализирани с Razor, което позволява динамично вмъкване на данни от C# в HTML шаблони.

Компонентът Controller приема заявките на потребителя, взаимодейства с моделите и избира кой изглед да върне като резултат. Например ProductsController зарежда продуктовия каталог и филтрите, CartController управлява количката, а OrdersController обработва checkout процеса и историята на поръчките. Именно контролерът осигурява логическата връзка между данните и визуализацията.

Предимството на MVC в един проект като „ЛапиХъб“ е, че позволява логиката на системата да бъде разделена на ясни, независими и поддържаеми части. Ако например е необходимо да се промени начинът, по който се визуализира каталога, това може да стане основно в изгледите. Ако трябва да се промени структурата на продукта или поръчката, това се прави в моделите. Ако трябва да се добави ново действие, то се имплементира в съответния контролер. Това разделение прави приложението по-устойчиво на промени и по-лесно за развитие.

2.3. ASP.NET Core MVC като платформа за уеб разработка

ASP.NET Core MVC е уеб фреймуърк на Microsoft, предназначен за изграждане на модерни, производителни и сигурни уеб приложения. Той стъпва върху .NET платформата и предоставя готов механизъм за маршрутизация, dependency injection, работа с middleware, валидация на входни данни и интеграция с други компоненти от екосистемата на Microsoft.

В проекта „ЛапиХъб“ ASP.NET Core MVC е основната технологична рамка, върху която е изградена цялата система. Чрез нея се реализира маршрутизирането към различните контролери и действия, обработката на GET и POST заявки, предаването на данни към Razor изгледите и защитата на действията чрез авторизация. По този начин приложението получава ясно структурирана и предвидима логика на изпълнение.

Едно от ключовите предимства на ASP.NET Core MVC е вградената система за dependency injection. Чрез нея контекстът на базата данни, UserManager, RoleManager и други услуги могат да бъдат инжектирани в контролерите, вместо да се създават ръчно. Това води до по-чист код и до по-добра тестируемост. В „ЛапиХъб“ тази възможност се използва постоянно – например за достъп до ApplicationDbContext или до идентификационните услуги на ASP.NET Identity.

Друг важен аспект е, че ASP.NET Core MVC работи много добре в комбинация с Razor Views. Това позволява изграждане на динамичен интерфейс, при който в HTML страниците могат да се визуализират данни от модела, да се използват условни блокове според ролята на потребителя и да се генерират форми, линкове и таблици с минимално количество помощен код. Тази комбинация е особено подходяща за онлайн магазин, където каталогът, количката и административните панели трябва да реагират на текущото състояние на системата.

2.4. Entity Framework Core и подходът Code First

Entity Framework Core е ORM технология, която позволява работа с база данни чрез обекти и LINQ заявки, вместо чрез ръчно писане на SQL за всяка операция. Това улеснява разработката, повишава четимостта на кода и намалява вероятността от грешки при стандартни CRUD операции. В „ЛапиХъб“ Entity Framework Core се използва като основен слой за достъп до данните.

Подходът Code First означава, че структурата на базата данни се описва първо чрез C# класове и конфигурация, а след това се създава физическата база данни чрез миграции. Този подход е особено удобен в учебни и дипломни проекти, защото позволява моделите и бизнес логиката да се развиват заедно. В разглеждания проект класовете Category, Product, CartItem, Order, OrderItem и ApplicationUser описват данните, а миграциите ги превръщат в реални таблици в SQL Server.

Миграциите имат съществена роля в жизнения цикъл на приложението. При промяна в модела може да се създаде нова миграция, която описва какво да се промени в базата данни. Това дава проследимост на развитието на проекта и улеснява пренасянето му на нов компютър или сървър. В дипломен контекст това е важно, защото документацията и кодът могат заедно да възстановят пълната работна среда.

Entity Framework Core осигурява и поддръжка на навигационни свойства и зареждане на свързани данни. В „ЛапиХъб“ това е особено важно, защото поръчките са свързани с потребители, елементи на поръчките и продукти, а продуктите от своя страна принадлежат към категории. Чрез Include и ThenInclude приложението може да зареди тези зависимости по контролиран начин и да визуализира пълната информация в детайлните страници.

2.5. SQL Server като релационна база данни

SQL Server е релационна система за управление на бази данни, която предлага надеждно съхранение на информация, поддръжка на външни ключове, индекси, ограничения и добри възможности за администриране. За проект като „ЛапиХъб“ това е естествен избор, тъй като електронната търговия се базира на ясно дефинирани обекти и връзки между тях.

Релационният модел е особено подходящ за онлайн магазин, тъй като позволява логически правилно представяне на зависимости. Един продукт принадлежи към една категория, един потребител може да има много поръчки, една поръчка съдържа много елементи, а всеки елемент на поръчката сочи към конкретен продукт. Тези отношения се реализират естествено чрез таблици, primary key и foreign key ограничения.

В проекта SQL Server съхранява както бизнес данните на магазина, така и таблиците, генерирани от ASP.NET Identity. Това означава, че в една и съща база са разположени данните за потребители, роли, категории, продукти, колички и поръчки. Така приложението има централизирано и последователно място за съхранение на цялата информация, необходима за неговата работа.

Предимство на SQL Server е и неговата добра интеграция с Entity Framework Core. Това позволява контекстът на приложението да работи надеждно с миграции, заявки и релации, без да е необходима допълнителна конфигурация извън стандартната .NET среда.

2.6. ASP.NET Core Identity – потребители, роли и сигурност

ASP.NET Core Identity представлява готова система за управление на потребители, вход, регистрация, роли и удостоверяване. Вместо тези механизми да се разработват от нулата, Identity предлага сигурна и стабилна основа, която може да бъде надграждана чрез custom потребителски модел и допълнителни полета. В проекта „ЛапиХъб“ именно Identity осигурява логиката за вход в системата и разграничаване на ролите.

Особено важно в разглежданото приложение е, че потребителският модел е разширен чрез custom класа ApplicationUser. Освен стандартните свойства, наследени от IdentityUser, в него са добавени собствени полета като FirstName и LastName, както и изчисляемо свойство FullName. Това позволява в системата да се

работи с по-пълна информация за клиента и да се реализира по-персонализирано потребителско преживяване.

Ролевият модел е централна част от приложението. Логически системата различава гост, регистриран потребител и администратор. На техническо ниво удостоверените роли са User и Admin, като те се присвояват и проверяват чрез механизма на Identity. Чрез атрибути като [Authorize] и [Authorize(Roles = "Admin")] различните действия и контролери могат да бъдат защитени спрямо необходимото ниво на достъп.

Използването на Identity има и сигурностен аспект. Паролите не се съхраняват в открит вид, а се обработват и съхраняват чрез защитения механизъм на платформата. Освен това страниците за регистрация и вход са интегрирани в цялостната логика на системата и могат да бъдат персонализирани според нуждите на проекта. В „ЛапиХъб“ това е използвано за въвеждане на български текстове, валидации и бизнес правила при регистрацията.

2.7. Потребителски интерфейс, Razor Views и Bootstrap 5

Потребителският интерфейс е пряко свързан с качеството на взаимодействието между системата и крайния потребител. При електронна търговия той не е само визуална обвивка, а ключов фактор за това дали потребителят ще може бързо да намери желания продукт, да разбере информацията за него и да завърши покупката без затруднения. Ето защо интерфейсът на „ЛапиХъб“ е изграден с акцент върху ясна структура, добра четимост и последователност между различните страници.

Razor Views предоставят механизъм за създаване на HTML изгледи, в които могат да се вмъкват динамични данни от C# кода. Това позволява каталозите, таблиците, формите и панелите да се изграждат динамично според текущото съдържание на базата данни и ролята на потребителя. Например в навигацията се визуализират различни връзки за гост, за логнат потребител и за администратор.

Bootstrap 5 се използва за реализиране на responsive и визуално подреден дизайн. Чрез неговите компоненти и grid система са изградени навигационната лента, картите за продукти, таблиците в администрацията, формите за регистрация, количката и страниците за поръчки. Подходът е особено подходящ за дипломен проект, тъй като позволява бързо и надеждно изграждане на професионално изглеждащ интерфейс без излишно усложняване на фронтенд частта.

В „ЛапиХъб“ интерфейсът е съобразен и с тематиката на магазина. Използван е приятелски и ненатрапчив визуален стил, който създава асоциация с продукти за домашни любимци, но в същото време остава достатъчно изчистен и професионален. Това е важно, защото добрият интерфейс трябва едновременно да създава характер на системата и да не пречи на нейната функционалност.

Глава 3. Анализ на изискванията към системата „ЛапиХъб“

Преди да бъде реализирана една софтуерна система, е необходимо ясно да бъдат определени нейните цели, потребители, ограничения и очаквани функции. Анализът на изискванията е етапът, който превръща общата идея за приложение в конкретен план за реализация. При проект като „ЛапиХъб“ той е особено важен, защото системата трябва да съчетае потребителски удобства, бизнес логика и административен контрол в една обща уеб платформа.

В настоящата глава се разглеждат основните изисквания към проекта. Те включват определяне на целевите потребители, описание на ролите в системата, формулиране на функционалните изисквания и извеждане на основните нефункционални характеристики. Така се създава рамката, върху която по-късно стъпват конкретните модели, контролери и изгледи.

3.1. Предназначение на системата и целева аудитория

„ЛапиХъб“ е предназначен да функционира като електронен магазин за продукти за домашни любимци. Основният бизнес сценарий е крайният клиент да може да открие нужната стока, да получи достатъчно информация за нея, да я добави в количка и да създаде поръчка. Поради това системата е ориентирана към удобна навигация, ясна организация на каталога и ограничаване на броя стъпки до финализиране на поръчката.

Целевата аудитория включва собственици на домашни любимци, които търсят продукти като храни, лакомства, играчки, аксесоари и средства за грижа. Тези потребители не винаги имат технически познания, затова интерфейсът трябва да бъде максимално ясен. От гледна точка на функционалността това означава използване на разбираеми бутони, видими навигационни елементи, добри съобщения при успех или грешка и логично структурирани страници.

Отделна група потребители са администраторите на системата. За тях приложението трябва да осигурява контрол върху съдържанието и бизнес данните. Това налага наличие на защитена администрация, чрез която да могат да се управляват категориите, продуктите, поръчките и потребителските акаунти. Следователно още на ниво изисквания системата трябва да бъде разглеждана като двукомпонентна: клиентска част и административна част.

3.2. Потребителски роли и нива на достъп

Системата е проектирана около три основни логически роли: гост, регистриран потребител и администратор. Това разделение е необходимо, за да се осигури едновременно свободен достъп до каталога и защитен достъп до чувствителни действия като поръчки и административно управление.

Гостът е всеки потребител, който посещава сайта без да е влязъл в системата. За него трябва да бъдат достъпни началната страница, каталогът с продукти, страницата с детайли за даден продукт, търсенето и филтрирането. Гостът има право да разглежда, но не и да извършва операции, които променят данни в системата – например добавяне в количка с постоянен потребителски контекст или финализиране на поръчка.

Регистрираният потребител е потребител, който има акаунт и е влязъл успешно в системата. За тази роля се изисква достъп до потребителско табло, количка, checkout страница, история на поръчките и детайли за собствените поръчки. Тази роля трябва да бъде ограничена само до личните данни на потребителя и никога да няма достъп до чужди поръчки или административни функции.

Администраторът е роля с разширени права. Освен всички възможности на обикновения потребител, администраторът трябва да разполага с достъп до административна навигация и панели за управление. Това включва преглед, създаване, редакция и изтриване на категории и продукти, преглед и изтриване на поръчки, както и преглед, редакция и контрол върху потребителите. Административните действия задължително трябва да бъдат защитени с ролеви ограничения.

За по-ясно обобщение правата на отделните роли могат да бъдат представени в следната таблица:

Функционалност	**Гост**	**Потребител**	**Администратор**
Разглеждане на	Да	Да	Да

начална страница и каталог			
Търсене, филтриране и сортиране	Да	Да	Да
Отваряне на детайли за продукт	Да	Да	Да
Регистрация и вход	Да	Не е необходимо	Не е необходимо
Добавяне в количка	Само след вход	Да	Да
Промяна на количество в количката	Не	Да	Да
Създаване на поръчка	Не	Да	Да
Преглед на лични поръчки	Не	Да	Да
Административен панел	Не	Не	Да
CRUD за продукти и категории	Не	Не	Да
Управление на потребители	Не	Не	Да

3.3. Функционални изисквания към публичната част

Публичната част на системата трябва да представя магазина по структуриран и удобен начин. Началната страница има представителна функция – тя въвежда потребителя в темата на магазина, представя основната идея и насочва вниманието към каталога. В проекта това е реализирано чрез начална страница с carousel, тематични секции и ясни връзки към продуктите.

Каталогът трябва да бъде достъпен без нужда от регистрация. Това е важно, защото електронният магазин трябва да позволява свободно разглеждане на

асортимента, преди потребителят да вземе решение дали да се регистрира. Продуктовата страница трябва да поддържа списъчен изглед с карти, снимка, име, марка, категория и цена, както и връзка към детайли.

Особено важни са възможностите за търсене и филтриране. Тъй като каталогът може да съдържа много продукти, потребителят трябва да може да стесни резултатите по критерии като име, марка, категория и ценови диапазон. В „ЛапиХъб“ е реализирано и сортиране по име и цена, което допълнително подобрява ориентацията в каталога.

Страницата с детайли за продукта трябва да съдържа по-пълна информация – голяма снимка, марка, категория, цена, описание и бутон за добавяне в количка. Освен това е полезно да се покажат и подобни продукти от същата категория, което увеличава вероятността потребителят да разгледа повече артикули.

3.4. Функционални изисквания към потребителската част

След вход в системата потребителят трябва да получи достъп до собствено табло, което обединява основната информация за профила му и служи като отправна точка към количката и поръчките. Това табло има навигационна функция и позволява на потребителя бързо да премине към най-често използваните си действия.

Количката е една от централните функционалности на всеки онлайн магазин. Изискванията към нея включват възможност за добавяне на продукт, увеличаване на количество при повторно добавяне, промяна на количеството в диапазон от 1 до 10 и премахване на продукт. Освен това системата трябва да изчислява междинната стойност за всеки ред и общата стойност за цялата количка.

Checkout процесът трябва да бъде максимално кратък и ясен. За целите на проекта е избрана опростена форма, която включва адрес за доставка, телефон и метод на плащане. Поръчката се финализира чрез създаване на запис в таблицата за поръчки и съответните записи в таблицата за елементите на поръчката, след което количката се изчиства.

Потребителят трябва да има и възможност да преглежда своите минали поръчки. Това означава наличие на списък с поръчки, подредени по дата, и детайлна страница за всяка поръчка. Съществено изискване е потребителят да вижда единствено собствените си поръчки, което е пряко свързано със сигурността на системата.

3.5. Функционални изисквания към административната част

Административната част трябва да предоставя на администратора достатъчно инструменти за поддържане на системата в работоспособно и актуално състояние. Това означава наличие на отделно административно табло и панели за управление на основните данни.

Управлението на категориите включва създаване, преглед, редактиране и изтриване на категории. При изтриване трябва да се вземе предвид дали към дадена категория има свързани продукти. Ако такива съществуват, системата трябва да блокира изтриването или да информира администратора за зависимостта.

Управлението на продуктите е най-съществената част от администрацията. Администраторът трябва да може да създава нов продукт, да редактира неговото име, марка, категория, цена, описание и URL на снимка, както и да изтрива продукт. Системата трябва да реагира контролирано, ако даден продукт участва в поръчки и не може да бъде премахнат без последствия.

Управлението на поръчките включва общ списък с всички поръчки, преглед на детайли и възможност за изтриване. Макар в реална система често да се използват статуси и обработка на жизнения цикъл на поръчката, за целите на дипломния проект е достатъчно администраторът да разполага с обзор и контрол върху данните.

Управлението на потребителите включва списък на всички регистрирани акаунти, детайли, редакция на основни данни и изтриване. При това е важно да се предвидят ограничения – например да не бъде позволено администраторът да изтрие собствения си акаунт по невнимание, както и да не се допуска изтриване на потребител с вече съществуващи поръчки.

3.6. Нефункционални изисквания

Освен функционалните изисквания, системата трябва да отговаря и на набор от нефункционални характеристики. Те описват не какво прави приложението, а как го прави. В дипломен проект тези изисквания са важни, защото показват качеството на инженерното решение.

На първо място стои изискването за добра структурираност и поддръжка на кода. Използването на MVC архитектура, отделни модели, view model-и и контролери има за цел да осигури ясно разделение на отговорностите. Това улеснява както разработката, така и бъдещото разширяване на приложението.

Следващ важен аспект е сигурността. Необходимо е чувствителните действия да бъдат защитени чрез авторизация, ролева проверка и правилно ограничение на достъпа. Потребителят трябва да вижда само собствените си поръчки, а административните действия трябва да бъдат достъпни само за роля Admin.

Интерфейсът трябва да бъде четим, последователен и удобен за работа. Това означава използване на ясни форми, таблици, карти, бутони и съобщения за грешка или успех. Поради изискването системата да бъде на български език всички текстове, етикети, валидиращи съобщения и бутони трябва да бъдат представени на български.

Накрая, приложението трябва да бъде лесно за стартиране и пренасяне. Чрез миграции, seed данни и ясна конфигурация в Program.cs проектът трябва да може да бъде възстановен и пуснат на нова среда с минимални допълнителни настройки.

3.7. Основни сценарии на използване

За да се провери дали изискванията са правилно формулирани, е полезно те да се разгледат и като потребителски сценарии. Един основен сценарий е следният: гост влиза в сайта, разглежда каталога, филтрира продукти по категория или марка, отваря детайли за конкретен продукт и решава да го добави в количката. Тъй като не е логнат, системата го пренасочва към вход или регистрация. След успешен вход продуктът може да бъде добавен и потребителят продължава към checkout.

Втори основен сценарий е работата на регистриран потребител. Той влиза в профила си, разглежда каталога, добавя няколко продукта, променя количествата в количката и потвърждава поръчка. След това проверява в секцията „Моите поръчки“, че новата поръчка е налична и детайлите ѝ са коректно записани.

Трети важен сценарий е административната поддръжка. Администраторът влиза в системата, отваря админ таблото, преглежда статистиката, управлява категории и продукти, следи поръчките и при необходимост редактира потребителска информация. Този сценарий показва, че приложението не е само витрина, а действително управляема електронна търговска система.

Глава 4. Проектиране и реализация на системата „ЛапиХъб“

Третата глава представя същинската практическа реализация на проекта. Тук се разглеждат структурата на приложението, конфигурацията на средата, моделите на данните, ролите, логиката на контролерите и изгледите, чрез които системата се визуализира пред различните потребители. Тази част е централна за дипломната документация, защото показва как теоретичните принципи от предходните глави са приложени в конкретен работещ софтуерен продукт.

При проектирането на „ЛапиХъб“ е следван принципът за ясна, предвидима и поддържаема структура. За тази цел логиката е разделена между Controllers, Models, Views, Data и Areas/Identity. Освен това са използвани view model-и, които улесняват пренасянето на данни към изгледите и позволяват формите и списъчните страници да работят с точно определен набор от свойства.

В следващите подточки са разгледани както общата организация на проекта, така и отделните модели и контролери. Особено внимание е отделено на връзките между таблиците, на конфигурацията на Identity и на административната част, защото именно те реализират основната стойност на системата.

4.1. Обща структура на проекта

(виж приложение 3)

Проектът е организиран по стандартната за ASP.NET Core MVC схема, при която основните компоненти са разделени в самостоятелни папки и области. Тази организация позволява бързо ориентиране в кода и ясно разпределение на отговорностите между отделните части на приложението.

Папката Controllers съдържа контролерите, които обработват потребителските заявки и връщат съответните изгледи или резултати. В проекта тя включва както публични контролери, така и административни. Папката Models съдържа бизнес моделите, които описват основните обекти в системата и техните зависимости. Папката ViewModels съдържа помощни класове за конкретни екрани и форми, например каталога, количката, checkout формата и административните списъци.

Папката Views съдържа Razor изгледите, организирани по контролери. Там се намират страниците за начална страница, каталог, детайли за продукт, табло, количка, поръчки и административни панели. Папката Data съдържа

ApplicationDbContext и помощен клас за seed на начални роли и администратор. Areas/Identity съдържа scaffold-натите страници за регистрация, вход и изход, които са персонализирани спрямо нуждите на проекта.

Тази структура е подходяща не само за учебен проект, но и за реално приложение, тъй като улеснява надграждането. При нужда от добавяне на нов модул могат да се създадат нов модел, контролер, view model и изгледи без да се нарушава съществуващата организация.

Общата логическа структура на проекта може да бъде обобщена в следната таблица:

Компонент	**Предназначение**
Controllers	Обработват HTTP заявките и управляват логиката между моделите и изгледите.
Models	Описват основните бизнес обекти и структурата на данните.
ViewModels	Съдържат класове за пренасяне на данни към конкретни екрани и форми.
Views	Razor изгледи за визуализация на интерфейса.
Data	Конфигурация на ApplicationDbContext, seed логика и връзка с базата данни.
Areas/Identity	Персонализирани страници за регистрация, вход, изход и Identity логика.
wwwroot	Статични файлове, ако са необходими – изображения, скриптове, стилове.

4.2. Конфигурация на приложението и ролята на Program.cs

(виж приложение 14)

В ASP.NET Core файлът Program.cs е входната точка на приложението. Той отговаря за конфигурирането на услугите, middleware pipeline-а и стартирането на системата. В проекта „ЛапиХъб“ именно в този файл се изгражда основната

инфраструктура – връзка към базата данни, интеграция на Identity, конфигурация на MVC, разрешаване на Razor Pages и seed на ролите и началния администратор.

Първата важна стъпка е извличането на connection string от appsettings.json и конфигурирането на ApplicationDbContext да използва SQL Server. Това позволява на приложението да работи с реална база данни и да използва миграции. Веднага след това се добавя Database Developer Page Exception Filter, който подпомага разработката и диагностицирането на проблеми в среда за разработка.

Следва конфигурацията на Identity. В проекта е използван AddDefaultIdentity<ApplicationUser>(), като върху стандартната конфигурация са зададени собствени правила за парола. Минималната дължина е 4 символа, а останалите стандартни изисквания за цифри, главни букви и специални символи са изключени, тъй като проектът е образователен и използва опростен бизнес сценарий. Към Identity е добавена и поддръжка на роли чрез AddRoles<IdentityRole>().

В Program.cs се конфигурира и обслужването на контролери с изгледи чрез AddControllersWithViews(), както и Razor Pages чрез MapRazorPages(). Това е важно, защото MVC частта на приложението работи чрез контролери и изгледи, а страниците за регистрация и вход принадлежат към Identity и се реализират като Razor Pages.

След изграждането на приложението се създава service scope, чрез който се извиква DbSeeder.SeedRolesAndAdminAsync(). Това е важна стъпка, защото гарантира, че при стартиране на системата в базата съществуват ролите Admin и User, както и поне един административен акаунт. Така приложението остава използваемо дори след първоначална инсталация на нова машина.

В middleware pipeline-а са включени стандартните елементи за HTTPS пренасочване, статични файлове, маршрутизация, автентикация и авторизация. Чрез това приложение получава необходимата последователност от стъпки за обработка на всяка заявка – от приемането ѝ, през проверка на достъпа, до връщане на HTML отговор.

Следният фрагмент показва в обобщен вид основните конфигурационни елементи в Program.cs:

```
builder.Services.AddDbContext<ApplicationDbContext>(options =>
options.UseSqlServer(connectionString));

builder.Services
```

```
.AddDefaultIdentity<ApplicationUser>(options =>
{
options.SignIn.RequireConfirmedAccount = false;
options.Password.RequiredLength = 4;
options.Password.RequireDigit = false;
options.Password.RequireLowercase = false;
options.Password.RequireUppercase = false;
options.Password.RequireNonAlphanumeric = false;
})
.AddRoles<IdentityRole>()
.AddEntityFrameworkStores<ApplicationDbContext>();

builder.Services.AddControllersWithViews();
```

Реално този код показва три ключови решения: използване на SQL Server чрез Entity Framework Core, използване на разширения потребителски модел ApplicationUser и включване на ролева логика. Те са достатъчни, за да изградят основата на приложението.

4.3. ApplicationDbContext и конфигурация на данните

(виж приложение 15)

ApplicationDbContext е класът, чрез който приложението комуникира с базата данни. Той наследява IdentityDbContext<ApplicationUser>, което означава, че освен собствените таблици на магазина, той включва и таблиците, необходими за Identity – потребители, роли, claims, логини и ролеви връзки. Това позволява бизнес данните и идентификационните данни да се съхраняват в обща база и да бъдат управлявани централизирано.

В ApplicationDbContext са дефинирани DbSet свойства за основните обекти на системата: Categories, Products, CartItems, Orders и OrderItems. Чрез тях Entity Framework Core създава съответните таблици и позволява изпълнение на заявки, добавяне на записи и проследяване на промени.

Съществена част от контекста е методът OnModelCreating(). В него чрез fluent API са описани релациите между таблиците, поведението при изтриване и някои ограничения за данните. Този метод е необходим, защото не всички зависимости могат или е удобно да бъдат описани само чрез атрибути в моделите.

Например продуктът е свързан с категория, като при опит за изтриване на категория с продукти се използва Restrict поведение. Това предпазва системата от неволно нарушаване на целостта на каталога. CartItem има връзка както към потребителя, така и към продукта, а чрез уникален индекс по двойката UserId и ProductId се гарантира, че един и същ потребител няма да има два отделни реда в количката за един и същ продукт.

При Order и OrderItem са зададени релации, които гарантират правилно поведение на историята на поръчките. Връзката Order → User е Restrict, за да не се позволява изтриване на потребител, ако съществуват свързани поръчки. Връзката OrderItem → Product също е Restrict, което е важен бизнес избор: ако един продукт е участвал в поръчка, не е добра идея историческите данни да бъдат счупени при последващо изтриване на продукта.

Освен релациите в контекста се съдържа и началното seed-ване на категориите и продуктите чрез HasData(). Това позволява системата да стартира с подготвен каталог и да бъде демонстрирана веднага след първоначално създаване на базата.

4.4. Потребителският модел ApplicationUser

(виж приложение 4)

ApplicationUser е разширеният потребителски модел на приложението. Той наследява стандартния IdentityUser и добавя допълнителни свойства, които са нужни за бизнес логиката на „ЛапиХъб“. В стандартния IdentityUser се съдържат основните идентификационни данни, например Id, UserName, Email и PhoneNumber. За целите на магазина обаче е необходимо към тях да се добавят име и фамилия, защото те се използват в потребителското табло, административния панел и визуализацията на клиентска информация в поръчките.

Използването на отделен клас ApplicationUser е добра практика, защото позволява системата да бъде персонализирана спрямо конкретния бизнес домейн, без да се нарушава стандартната Identity функционалност. В „ЛапиХъб“ това решение дава възможност за по-добра интеграция между модула за удостоверяване и останалата част от приложението.

Важна особеност е, че при регистрация новият потребител се добавя в роля User. Така веднага след създаване на акаунт той може да използва потребителските функции на магазина, включително количка и поръчки.

Основните свойства на модела са представени в следната таблица:

Свойство	**Тип**	**Предназначение**
Id	string	Уникален идентификатор на потребителя, наследен от Identity.
UserName	string	Потребителско име; в проекта съвпада с имейла.
Email	string	Имейл адрес за вход и контакт.
PhoneNumber	string	Телефонен номер на потребителя.
FirstName	string	Собствено име на потребителя.
LastName	string	Фамилия на потребителя.
FullName	string (computed)	Изчисляемо свойство, връщащо обединение на име и фамилия.
Orders	ICollection<Order>	Навигационно свойство към поръчките на потребителя.
CartItems	ICollection<CartItem>	Навигационно свойство към елементите в количката.

Първата група свойства – Id, UserName, Email и PhoneNumber – идват от базовия IdentityUser и осигуряват стандартната идентификационна логика. Втората група – FirstName, LastName и FullName – е специфична за проекта. FullName не се съхранява като отделна колона, а се изчислява динамично от другите две свойства. Това намалява дублирането на данни и поддържа логиката проста.

Навигационните свойства Orders и CartItems свързват потребителя с неговите поръчки и текуща количка. Тези зависимости са ключови за логиката на приложението, тъй като почти всяка операция в потребителската част минава през идентифициране на текущия ApplicationUser.

При регистрация системата създава нов ApplicationUser, задава въведените стойности за име, фамилия, телефон и имейл, а след успешно създаване на акаунта го добавя в роля User. Това може да се обобщи със следния фрагмент:

```
var result = await _userManager.CreateAsync(user, Input.Password);


if (result.Succeeded)
{
await _userManager.AddToRoleAsync(user, "User");
await _signInManager.SignInAsync(user, isPersistent: false);
}
```

Този механизъм гарантира, че регистрираният потребител веднага получава правилното ниво на достъп и не е необходимо допълнително ръчно присвояване на роли.

4.5. Модел Category

(виж приложение 6)

Category представлява категория на продуктите в магазина. Нейната основна роля е да групира продуктите по общ признак, така че каталогът да бъде по-лесен за навигация и филтриране. В „ЛапиХъб“ категориите отразяват основните продуктови групи – храна за кучета, храна за котки, лакомства, играчки, хигиена и грижа, аксесоари.

Категорията е една от най-простите, но и най-важни същности в системата. Чрез нея се организира каталожната структура на магазина и се реализира филтрирането по групи в продуктовата страница.

Основните свойства на модела са представени в следната таблица:

Свойство	Тип	Предназначение
Id	int	Уникален идентификатор на категорията.
Name	string	Име на категорията, визуализирано в каталога и администрацията.
Products	ICollection<Product>	Навигационно свойство към продуктите в категорията.

Полето Name е задължително и има валидиращо ограничение за максимална дължина. Това предпазва системата от празни или прекомерно дълги имена, които биха затруднили визуализацията.

Връзката между Category и Product е едно-към-много: една категория може да съдържа множество продукти, но всеки продукт принадлежи на точно една категория. В базата това се реализира чрез външен ключ CategoryId в таблицата Products.

В OnModelCreating() е зададено Restrict поведение при изтриване на категория. Това е логичен бизнес избор, защото не трябва да се допуска изтриване на категория, ако към нея все още има продукти. Така се гарантира целостта на каталога.

4.6. Модел Product

(виж приложение 11)

Product е централният бизнес модел в електронния магазин. Той представя конкретен продукт, който може да бъде визуализиран в каталога, добавян в количка и включван в поръчки. В тематиката на „ЛапиХъб“ това са продукти за домашни любимци – например различни видове храни, играчки, легла, каишки, шампоани и други принадлежности.

Поради важността на модела Product, той съдържа достатъчно информация, за да може потребителят да вземе решение за покупка. Това включва име, марка, категория, цена, описание и връзка към изображение.

Основните свойства на модела са представени в следната таблица:

Свойство	Тип	Предназначение
Id	int	Уникален идентификатор на продукта.
Name	string	Име на продукта.
Brand	string	Марка на продукта.
CategoryId	int	Външен ключ към категорията.
Category	Category	Навигационно свойство към категорията.
Price	decimal	Цена на продукта.
Description	string	Подробно описание на продукта.
ImageUrl	string	URL адрес към изображение на продукта.
CartItems	ICollection<CartItem>	Елементи от колички, които съдържат продукта.
OrderItems	ICollection<OrderItem>	Елементи от поръчки, в които участва продуктът.

Моделът съдържа валидиращи атрибути за задължителни полета и ограничения на дължината. Цената е представена като decimal и е ограничена в разумен диапазон чрез Range атрибут, за да се предотврати въвеждане на невалидни стойности.

CategoryId реализира връзката към Category. Това позволява продуктите да се групират по категории и да се филтрират в публичната част на каталога. Navigation свойството Category служи за лесно визуализиране на името на категорията в изгледите и администрацията.

Навигационните свойства CartItems и OrderItems са важни за историята на системата. Един продукт може едновременно да присъства в колички на различни потребители и да участва в множество поръчки. Поради тази причина при изтриване на продукт е необходимо внимателно да се отчете дали той вече е използван в историята на поръчките.

4.7. Модел CartItem

(виж приложение 5)

CartItem представлява единичен ред в количката на даден потребител. Вместо количката да бъде отделна сложна структура, тя се моделира като множество CartItem записи, всеки от които свързва потребител и продукт с конкретно количество. Това решение е практично и се използва често при онлайн магазини, защото позволява проста и ясна организация на данните.

Чрез CartItem системата може да пази текущото състояние на количката на всеки потребител между отделните заявки. Това е ключово, защото добавянето на продукти в количка е един от основните междинни етапи преди поръчката.

Основните свойства на модела са представени в следната таблица:

Свойство	**Тип**	**Предназначение**
Id	int	Уникален идентификатор на реда в количката.
UserId	string	Външен ключ към ApplicationUser.
User	ApplicationUser	Навигационно свойство към потребителя.
ProductId	int	Външен ключ към Product.
Product	Product	Навигационно свойство към продукта.
Quantity	int	Количество от продукта в количката.
TotalPrice	decimal (NotMapped)	Изчисляема стойност за общата цена на реда.

UserId и ProductId са двата основни външни ключа в този модел. Те определят кой потребител е добавил кой продукт. Количеството е ограничено между 1 и 10, което е бизнес правило, приложено както в модела, така и в интерфейса.

Особено важно е, че в ApplicationDbContext е дефиниран уникален индекс по двойката UserId и ProductId. Това означава, че един потребител не може да има два отделни реда в количката за един и същ продукт. Вместо това, при повторно

добавяне количеството се увеличава. Това поведение улеснява интерфейса и предотвратява излишно дублиране.

TotalPrice е отбелязано като NotMapped, защото не се записва в базата данни. То се изчислява динамично чрез цената на свързания продукт и количеството в реда. Така се избягва съхранение на излишни данни и се намалява рискът от несъответствие.

4.8. Модел Order

(виж приложение 7)

Order представлява основната информация за една завършена поръчка. Това е така наречената „глава“ на поръчката – тя съдържа общите данни за клиента, адреса, телефона, начина на плащане, статуса, датата и общата стойност.

В практиката поръчката се разглежда като отделен документ, който обединява множество OrderItem записи. Затова моделът Order има не само самостоятелни свойства, но и колекция от елементи на поръчката.

Основните свойства на модела са представени в следната таблица:

Свойство	**Тип**	**Предназначение**
Id	int	Уникален номер на поръчката.
UserId	string	Външен ключ към ApplicationUser.
User	ApplicationUser	Потребителят, който е направил поръчката.
Address	string	Адрес за доставка.
PhoneNumber	string	Телефон за контакт при доставката.
PaymentMethod	enum PaymentMethod	Избран начин на плащане.
Status	enum OrderStatus	Текущ статус на поръчката.
TotalAmount	decimal	Обща сума на поръчката.
CreatedOn	DateTime	Дата и час на създаване.
OrderItems	ICollection<OrderItem>	Елементите, включени в

		поръчката.

Външният ключ UserId свързва поръчката с нейния собственик. Това е основата, върху която се реализира потребителската защита при преглед на лични поръчки. Когато в контролера се търсят поръчки за текущия потребител, именно по това поле се филтрират резултатите.

Полетата Address, PhoneNumber и PaymentMethod се попълват от checkout формата. В проекта плащането е реализирано симулирано чрез наложен платеж, което е достатъчно за учебна система и елиминира нуждата от интеграция с външен платежен оператор.

Status позволява поръчката да има текущо състояние. Макар в проекта администраторът да не променя статуса динамично, самото наличие на такова поле е правилно архитектурно решение, защото позволява бъдещо разширяване на системата.

Колекцията OrderItems съдържа конкретните продукти и количества, включени в поръчката. По този начин Order и OrderItem работят като класическа схема master-detail.

4.9. Модел OrderItem

(виж приложение 8)

OrderItem представлява единичен продукт в рамките на конкретна поръчка. Ако Order съдържа общите данни за документа, то OrderItem съдържа редовете в този документ – кой продукт е поръчан, в какво количество и на каква единична цена.

Този модел е важен, защото позволява една поръчка да съдържа повече от един продукт. Без него би било невъзможно да се представи коректно съдържанието на поръчката.

Основните свойства на модела са представени в следната таблица:

Свойство	**Тип**	**Предназначение**
Id	int	Уникален идентификатор на реда в поръчката.
OrderId	int	Външен ключ към Order.
Order	Order	Навигационно свойство

		към поръчката.
ProductId	int	Външен ключ към Product.
Product	Product	Навигационно свойство към продукта.
Quantity	int	Поръчано количество от продукта.
Price	decimal	Единична цена на продукта към момента на поръчката.

Най-важният архитектурен избор при OrderItem е съхраняването на Price като отделно поле. Причината е, че цената на продукта може да се промени след време, но историческата поръчка трябва да остане вярна на стойността към момента на създаването ѝ. Ако системата използваше само текущата цена от Product, старите поръчки биха станали неточни.

Връзката към Order реализира принадлежността на реда към конкретна поръчка, а връзката към Product позволява по-късно да се визуализира името, снимката и марката на продукта в детайлите на поръчката.

Чрез този модел се осигурява правилна история на продажбите и детайлно проследяване на съдържанието на всяка заявка.

4.10. Контролери и основна бизнес логика

Контролерите в проекта изпълняват ролята на основен посредник между потребителските заявки, моделите и изгледите. Чрез тях се реализира по-голямата част от бизнес логиката на системата – зареждане на каталога, обработка на количката, checkout, извличане на поръчки, както и административните операции. В тази подточка всеки контролер е разгледан отделно с неговото предназначение, основни действия и логически сценарии на работа.

За по-голяма яснота контролерите могат да се разделят на три групи: публични контролери, потребителски контролери и административни контролери. Първата група обслужва страниците, достъпни за всички посетители. Втората изисква удостоверен потребител и обхваща таблото, количката и поръчките. Третата

е защитена чрез роля Admin и реализира управлението на съдържанието и потребителите.

4.10.1. HomeController

HomeController е контролерът, който обслужва общите страници на приложението – начална страница, контакти и стандартната страница за грешка. Неговата логика е сравнително проста, но ролята му е важна, защото именно чрез него потребителят влиза в системата и получава първоначална представа за магазина.

Началната страница на „ЛапиХъб“ има представителен характер. Тя не просто показва статично съдържание, а изпълнява и навигационна функция чрез бутоните към каталога. Контролерът подава към нея основния входен маршрут на системата.

Основните действия на контролера са представени в следната таблица:

Действие	**HTTP**	**Достъп**	**Предназначение**
Index	GET	Публичен	Връща началната страница на магазина.
Contacts	GET	Публичен	Връща страницата с контактна информация.
Error	GET	Публичен	Връща стандартен изглед за грешка.

Действието Index няма сложна бизнес логика, но е важно като стартова точка на приложението. То връща визуално оформената начална страница с carousel и представяне на магазина.

Действието Contacts служи за отделяне на контактната информация в самостоятелна страница, вместо тя да бъде смесена с други общи изгледи. Това решение подобрява навигацията и прави сайта по-завършен.

Действието Error използва стандартен ErrorViewModel и предоставя централизирано място за визуализиране на грешки. Макар в учебен проект това да не е сложен модул, наличието му е важна част от добрата архитектурна практика.

4.10.2. ProductsController

ProductsController е един от най-важните контролери в системата, защото реализира каталога на магазина. Чрез него потребителят достъпва списъка с продукти, използва търсене и филтри, разглежда детайли и инициира добавяне към количката.

Този контролер работи едновременно с публична и защитена логика. Каталогът и детайлите са публично достъпни, докато добавянето в количката изисква потребителят да бъде логнат.

Основните действия на контролера са представени в следната таблица:

Действие	**HTTP**	**Достъп**	**Предназначение**
Index	GET	Публичен	Показва каталога с поддръжка на търсене, филтриране и сортиране.
Details	GET	Публичен	Показва детайлна информация за даден продукт.
AddToCart	POST	Удостоверен потребител	Добавя продукта в количката на текущия потребител.

Действието Index приема параметри като searchTerm, categoryId, brand, minPrice, maxPrice и sortBy. Те се използват за изграждане на LINQ заявка върху Products, която динамично се променя според избраните филтри. Това решение е гъвкаво и позволява комбиниране на няколко критерия едновременно.

Освен списъка с продукти, действието Index подготвя и допълнителни данни за изгледа: списък с категории, списък с марки и статистика за броя продукти по марки чрез GroupBy. По този начин един контролер подава към изгледа всички данни, нужни за sidebar филтрите и за основната grid визуализация.

Действието Details зарежда избрания продукт заедно с неговата категория и извлича до четири подобни продукта от същата категория. Това увеличава полезността на страницата и наподобява поведение на реални онлайн магазини.

Действието AddToCart е реализирано като POST, което е правилно от гледна точка на уеб сигурността и семантиката на HTTP. То първо проверява текущия потребител, след това намира продукта и търси дали вече съществува ред в количката за тази комбинация от потребител и продукт. Ако няма такъв ред, създава нов CartItem, а ако има – увеличава количеството до максимум 10. Това е много важен елемент от бизнес логиката на магазина.

Функционалният поток, реализиран от контролера, може да се обобщи така: потребителят отваря каталога, стеснява резултатите чрез филтри, разглежда конкретен продукт и го изпраща към количката. Ако не е логнат, достъпът до това действие се прекъсва чрез авторизация и системата го пренасочва към вход.

4.10.3. DashboardController

DashboardController обслужва личното табло на регистрирания потребител. Неговата роля е да представя в концентриран вид основните лични данни и връзки към най-важните потребителски функции – количка и поръчки.

Това е сравнително прост контролер, но е важен от гледна точка на потребителското преживяване, защото обединява персоналната част на магазина в едно начално място.

Основните действия на контролера са представени в следната таблица:

Действие	**HTTP**	**Достъп**	**Предназначение**
Index	GET	Удостоверен потребител	Връща потребителското табло с лични данни и връзки към количка и поръчки.

Контролерът използва UserManager<ApplicationUser>, за да извлече текущия потребител. След това тези данни се преобразуват във ViewModel, който съдържа само свойствата, нужни за изгледа – име, фамилия, имейл и телефон.

Използването на отделен DashboardViewModel е правилно решение, защото не е необходимо изгледът да работи директно с целия ApplicationUser. Така се постига по-добра изолация и по-ясен интерфейс между контролера и изгледа.

4.10.4. CartController

CartController реализира логиката на количката – една от най-важните части във всеки онлайн магазин. Той е достъпен само за логнати потребители, защото съдържанието на количката се свързва с конкретен потребителски акаунт.

Количката трябва да предоставя не само списък с продуктите, но и възможност за промяна на количеството, премахване на артикули и визуализиране на общата сума.

Основните действия на контролера са представени в следната таблица:

Действие	**HTTP**	**Достъп**	**Предназначение**
Index	GET	Удостоверен потребител	Показва съдържанието на количката на текущия потребител.
UpdateQuantity	POST	Удостоверен потребител	Променя количеството на конкретен ред в количката.
Remove	POST	Удостоверен потребител	Премахва продукт от количката.

Действието Index зарежда всички CartItem записи за текущия потребител, включва свързаните продукти и ги преобразува във ViewModel за визуализация. Това позволява в изгледа да се покажат снимка, име, марка, единична цена, количество и обща сума за всеки ред.

UpdateQuantity е реализирано като POST действие с anti-forgery защита. Контролерът валидира новата стойност, търси съответния CartItem по идентификатор и потребител и записва новото количество. Това действие е пример за малка, но критична бизнес операция, защото чрез него се влияе върху стойността на бъдещата поръчка.

Remove е най-простото действие в контролера, но е важно за използваемостта. То дава възможност потребителят да коригира избора си преди финализиране на checkout процеса.

Цялостният поток тук е следният: от каталога продуктите постъпват в количката, след което потребителят коригира количествата и преминава към поръчка. Този поток е типичен за B2C електронна търговия и е реализиран по максимално ясен начин.

4.10.5. OrdersController

OrdersController обработва checkout процеса и историята на поръчките на потребителя. Това е контролерът, който превръща временния избор в количката в постоянен бизнес запис – завършена поръчка.

Логиката му включва както визуализация на минали поръчки, така и създаване на нови поръчки на база текущото съдържание на количката.

Основните действия на контролера са представени в следната таблица:

Действие	**HTTP**	**Достъп**	**Предназначение**
MyOrders	GET	Удостоверен потребител	Показва списъка с поръчките на текущия потребител.
Details	GET	Удостоверен потребител	Показва детайлите на конкретна поръчка, ако тя принадлежи на текущия потребител.
Checkout	GET	Удостоверен потребител	Зарежда checkout формата и обобщението на количката.
Checkout	POST	Удостоверен потребител	Създава поръчка и елементите ѝ, след което изчиства количката.

Действието MyOrders извлича поръчките на текущия потребител, сортира ги по дата и ги преобразува в списъчен view model. Това позволява визуализиране на кратка история с номер на поръчка, дата, статус, брой артикули и обща сума.

Действието Details съдържа важна защитна логика. То търси поръчка не само по Id, но и по UserId на текущия потребител. По този начин се гарантира, че потребителят не може да отвори чужда поръчка просто чрез промяна на идентификатора в URL адреса. Това е критичен елемент от сигурността на системата.

Checkout GET подготвя формата за поръчка. Ако количката е празна, контролерът връща потребителя обратно към количката със съобщение. Ако в количката има артикули, се подготвят адрес, телефон и списък с продукти за обобщение.

Checkout POST е същинската бизнес операция по създаване на поръчка. След повторно извличане на артикулите от количката, контролерът създава нов Order обект, изчислява общата сума, добавя OrderItem записи за всеки CartItem и накрая премахва редовете от количката. Това е най-важният транзакционен момент в приложението.

Функционалният поток може да се обобщи така: количка → checkout форма → запис в Orders → запис в OrderItems → изчистване на количката → пренасочване към история на поръчките. Реализиран по този начин, процесът е ясен, предвидим и лесен за проследяване.

4.10.6. AdminController

AdminController обслужва административното табло на системата. Неговата функция е да даде обобщена картина на магазина – брой продукти, категории, поръчки и потребители. Макар да не съдържа сложна бизнес логика, този контролер има важно навигационно значение за администратора.

Достъпът до него е ограничен чрез [Authorize(Roles = "Admin")], което гарантира, че само потребители с роля Admin могат да достигат административното табло.

Основните действия на контролера са представени в следната таблица:

Действие	**HTTP**	**Достъп**	**Предназначение**
Index	GET	Администратор	Показва

			административното табло и броя на основните обекти в системата.

Контролерът извършва агрегиращи заявки към базата данни и подава резултатите към view model, предназначен за статистически карти в изгледа. Това решение е просто, но ефективно, защото показва на администратора моментно състояние на системата.

4.10.7. AdminCategoriesController

AdminCategoriesController реализира CRUD логиката за категориите в магазина. Неговата задача е да позволява създаване, редактиране, преглед и изтриване на категории от административния панел.

Този контролер е показателен пример за типичен административен CRUD модул: използва view model-и, валидира входни данни, връща съобщения за успех или грешка и защитава действията чрез администраторска роля.

Основните действия на контролера са представени в следната таблица:

Действие	**HTTP**	**Достъп**	**Предназначение**
Index	GET	Администратор	Показва списък с всички категории и броя продукти в тях.
Details	GET	Администратор	Показва детайли за избрана категория.
Create	GET/POST	Администратор	Създава нова категория.
Edit	GET/POST	Администратор	Редактира съществуваща категория.
Delete	GET	Администратор	Показва страница за потвърждение на изтриване.
DeleteConfirmed	POST	Администратор	Изтрива категория,

			ако няма свързани продукти.

При Create и Edit контролерът извършва проверка дали не съществува друга категория със същото име. Това е важно, защото предотвратява дублиране в каталожната структура и объркване при филтриране и визуализация.

При DeleteConfirmed контролерът проверява дали категорията има свързани продукти. Ако такива съществуват, изтриването се блокира. Това е правилно поведение както на бизнес, така и на техническо ниво, защото запазва целостта на каталога.

Този контролер илюстрира принципа, че административните действия не бива просто да пишат в базата, а трябва да пазят системата от нелогични и разрушителни операции.

4.10.8. AdminProductsController

AdminProductsController е най-съществената административна част на магазина, защото чрез него се управлява основното съдържание – продуктовият каталог. Контролерът позволява на администратора да създава, редактира, разглежда и изтрива продукти.

Поради по-големия обем данни при продуктите тук се използват и по-богати view model-и, включително избор на категория чрез dropdown и работа с описание и URL на изображение.

Основните действия на контролера са представени в следната таблица:

Действие	**HTTP**	**Достъп**	**Предназначение**
Index	GET	Администратор	Показва списък с всички продукти.
Details	GET	Администратор	Показва подробности за избран продукт.
Create	GET/POST	Администратор	Създава нов продукт.
Edit	GET/POST	Администратор	Редактира данни за продукт.
Delete	GET	Администратор	Показва

			потвърждение за изтриване.
DeleteConfirmed	POST	Администратор	Премахва продукт, ако това е позволено от зависимостите.

При Create и Edit контролерът първо зарежда списък с категории, който се подава към формата. Това е необходимо, защото всеки продукт задължително трябва да принадлежи към валидна категория. По този начин се избягва въвеждане на неконсистентни данни.

При запис на продукта се валидират всички основни свойства – име, марка, категория, цена, описание и снимка. Тази информация е достатъчна, за да може продуктът да бъде пълноценно използван както в каталога, така и в checkout и административните екрани.

DeleteConfirmed е реализирано внимателно, тъй като продуктът може да участва в колички и поръчки. В случай на зависимост от поръчки контролерът обработва възможната грешка и връща съобщение към администратора. Това е добър пример как бизнес логиката и зависимостите в базата данни влияят пряко върху поведението на интерфейса.

4.10.9. AdminOrdersController

AdminOrdersController осигурява административен достъп до всички поръчки в системата. Докато обикновеният потребител може да вижда само собствените си поръчки, администраторът трябва да има глобален обзор върху всички заявки.

Контролерът е предназначен основно за преглед и контрол, а не за сложна обработка на жизнения цикъл на поръчката. Това съответства на целите на дипломния проект.

Основните действия на контролера са представени в следната таблица:

Действие	**HTTP**	**Достъп**	**Предназначение**
Index	GET	Администратор	Показва списък с всички поръчки.
Details	GET	Администратор	Показва пълните детайли за избрана

			поръчка.
Delete	GET	Администратор	Показва потвърждение за изтриване на поръчка.
DeleteConfirmed	POST	Администратор	Изтрива поръчката и свързаните редове към нея.

Действието Index зарежда поръчките заедно със свързаните потребители и редове, за да може в списъчния изглед да се покажат клиент, дата, статус, брой артикули и сума. Това е типичен управленски екран, който дава на администратора обща картина на случващото се в магазина.

Детайлната страница на поръчката използва Include и ThenInclude, за да зареди клиента, редовете на поръчката и самите продукти. По този начин административният интерфейс може да покаже практически същата информация, която вижда и потребителят, но без ограничение до собствените записи.

При изтриване контролерът премахва първо елементите на поръчката, а след това самата поръчка. Това решение е съобразено с релационната структура на данните и е пример за внимателно управление на зависимости.

4.10.10. AdminUsersController

AdminUsersController осигурява административно управление на потребителските акаунти. Той предоставя списък с всички потребители, детайлна информация, редакция на основни данни и изтриване, когато това е допустимо.

Това е един от най-деликатните контролери в системата, защото работи с Identity потребители и изисква внимание към сигурността и консистентността на данните.

Основните действия на контролера са представени в следната таблица:

Действие	**HTTP**	**Достъп**	**Предназначение**
Index	GET	Администратор	Показва всички потребители, ролята им и брой поръчки/елементи

			в количка.
Details	GET	Администратор	Показва детайли за избран потребител.
Edit	GET/POST	Администратор	Редактира основни данни на потребителя.
Delete	GET	Администратор	Показва потвърждение за изтриване.
DeleteConfirmed	POST	Администратор	Изтрива потребител при позволени условия.

Index изгражда списъчен модел за всички потребители и чрез UserManager проверява дали даден потребител има роля Admin. Така в административната таблица ясно се различават обикновените акаунти от администраторските.

Edit позволява промяна на име, фамилия, имейл и телефон. Тъй като имейлът в системата се използва и като UserName, при редакция се актуализират и двете свойства, както и техните нормализирани стойности. Това е важно, за да не се наруши Identity логиката.

DeleteConfirmed съдържа няколко защитни проверки. Администраторът не може да изтрие собствения си акаунт, а потребител с вече съществуващи поръчки също не може да бъде премахнат, защото това би нарушило историята на магазина. Ако потребителят има само продукти в количка, те се премахват и след това акаунтът може да бъде изтрит.

Този контролер показва как административната логика трябва да бъде съчетана с предпазни механизми, които защитават системата от неволни или опасни операции.

4.11. Изгледи и потребителски интерфейс

Изгледите в проекта са реализирани чрез Razor и организирани по контролери. Това означава, че за всеки основен функционален модул – продукти,

количка, поръчки, администрация – съществува отделна група от .cshtml файлове. Тази организация улеснява поддръжката, защото логиката на контролера и визуалната структура на съответния екран се намират в предвидима папка.

Особено важен за приложението е общият layout файл (виж приложение 12). Чрез него се реализират навигационната лента, footer-ът, Bootstrap връзките и role-based менюто. Именно в _Layout.cshtml се определя кои елементи да се показват за гост, кои за логнат потребител и кои само за администратор. Това е ключов елемент от потребителското преживяване, защото позволява интерфейсът да бъде адаптивен спрямо текущия контекст.

Началната страница е реализирана като представителен изглед с carousel, обяснителни секции и призив за разглеждане на каталога. Тя няма сложна бизнес логика, но изпълнява маркетингова и навигационна функция. Продуктовият каталог е проектиран като двуколонен екран с филтриращ панел и grid от карти. Това улеснява прегледа на голям брой продукти и доближава приложението до реални e-commerce решения.

Страницата с детайли за продукт е организирана така, че да показва голяма снимка, основна информация, цена, описание и бутон за добавяне в количка. В долната ѝ част има секция с подобни продукти, което създава естествен навигационен преход в каталога.

Количката, checkout страницата и списъкът с поръчки са реализирани с по-практичен интерфейс – таблици, форми, обобщения и съобщения за успех/грешка. Тези изгледи са ориентирани към бързо извършване на конкретни действия, а не към представително съдържание. Същият принцип важи и за административните изгледи, където доминират таблици, бутони за CRUD операции и странична административна навигация.

Използването на Bootstrap 5 позволява изгледите да бъдат responsive и визуално последователни. Освен това приложението използва ясни празни състояния, информационни съобщения и потвърждения при изтриване, което подобрява използваемостта и намалява риска от неволни действия.

Основните групи изгледи в системата могат да бъдат обобщени така:

Група изгледи	**Примери**	**Предназначение**
Общи изгледи	Home/Index, Home/Contacts,	Начална визуализация, навигация и обща

	Shared/_Layout	структура.
Каталог и продукти	Products/Index, Products/Details	Разглеждане, търсене, филтриране и избор на продукти.
Потребителска част	Dashboard/Index, Cart/Index, Orders/Checkout, Orders/MyOrders, Orders/Details	Работа на логнат потребител с количка и поръчки.
Identity изгледи	Account/Login, Account/Register, Account/Logout	Регистрация, вход и изход от системата.
Административни изгледи	Admin/*, AdminProducts/*, AdminCategories/*, AdminOrders/*, AdminUsers/*	Управление на съдържание, поръчки и потребители.

4.12. Валидация на входните данни

Валидацията е задължителна част от всяка уеб система, която приема вход от потребителя. В „ЛапиХъб“ тя е реализирана на няколко нива. Първо, при моделите и view model-ите се използват DataAnnotations атрибути като Required, StringLength, Range, EmailAddress, Phone и Compare. Чрез тях се дефинират основните правила за допустимите стойности.

Второ, контролерите извършват допълнителни логически проверки, когато това е необходимо. Например при създаване или редакция на категория се проверява дали вече съществува категория със същото име. При checkout се проверява дали количката не е празна. При изтриване на потребител или категория се проверяват зависимостите с други данни.

Трето, самите изгледи използват asp-validation-for и asp-validation-summary, за да показват съобщенията по ясен и последователен начин. Важно изискване на проекта е всички текстове и валидации да бъдат на български език. Това е

реализирано чрез български ErrorMessage стойности в моделите и чрез персонализирани Identity страници за регистрация и вход.

Комбинацията от атрибутна, логическа и визуална валидация прави приложението по-надеждно и устойчиво на некоректен вход. Това е особено важно в e-commerce система, където невалидни данни могат да доведат до грешни поръчки, неработещи профили или проблеми при административното управление.

4.13. Роли, авторизация и сигурност

Сигурността в проекта е реализирана чрез ASP.NET Core Identity и атрибутно базирана авторизация. Всички действия, които променят данни в системата, са защитени по подходящ начин. Например количката и поръчките изискват удостоверен потребител, а административните панели са достъпни само за роля Admin.

Защитата на административните контролери се реализира чрез [Authorize(Roles = "Admin")]. Това е пряко и разбираемо решение, което ясно показва намерението в кода. Така няма риск публичните потребители да достигнат до CRUD логиката за продукти, категории, поръчки и потребители.

Потребителската сигурност се проявява и при работа с поръчките. Дори логнат потребител няма право да вижда чужди поръчки, което се гарантира чрез филтриране по текущия UserId в OrdersController. Това е типичен пример за object-level security – не е достатъчно страницата да е достъпна само за логнати потребители, а трябва да се провери и дали конкретният ресурс принадлежи на текущия потребител.

При POST форми в системата се използва anti-forgery защита чрез ValidateAntiForgeryToken и скрити anti-forgery токени в изгледите. Това е важна защита срещу CSRF атаки и представлява добра практика дори в учебни проекти.

От гледна точка на административната сигурност са предвидени и някои предпазни мерки в бизнес логиката. Администраторът не може да изтрие собствения си акаунт, категория с продукти не може да бъде изтрита, а потребител с поръчки също не може да бъде премахнат. Тези правила са примери за това, че сигурността не е само въпрос на вход и роли, а и на защита на целостта на бизнес данните.

Следният фрагмент илюстрира типичната защита на административен контролер:

```
[Authorize(Roles = "Admin")]
```

```
public class AdminProductsController : Controller
{
// административна логика
}
```

Подобни атрибути се използват последователно във всички административни контролери и са един от основните механизми за защита на приложението.

4.14. Начални данни и seed логика

(виж приложение 13)

За да бъде приложението използваемо веднага след първоначално създаване на базата данни, в проекта е реализирано начално seed-ване на данни. То обхваща две основни направления: роли и администраторски акаунт, както и начални категории и продукти.

Ролите Admin и User се създават чрез DbSeeder, който се извиква от Program.cs при стартиране на приложението. Ако липсват, те се добавят автоматично. След това се проверява дали съществува административен акаунт и ако не съществува, системата го създава. Това е удобно решение, защото позволява още след първото стартиране да се влезе в администрацията без допълнителни ръчни настройки.

Категориите и продуктите са seed-нати директно в ApplicationDbContext чрез HasData(). Така системата стартира с базов каталог и публичната част може да бъде демонстрирана веднага. За дипломна разработка това е особено полезно, защото позволява проектът да бъде показан в завършен вид още след изпълнение на миграциите.

Seed данните не са случайни – те са подбрани така, че да покриват различни категории и марки и да позволяват смислено тестване на филтрите, каталога и административния панел. По този начин началните данни играят роля както в демонстрацията на системата, така и в нейната техническа устойчивост при пренасяне.

4.15. Обобщение на основния функционален поток

За да се разбере цялостната логика на системата, е полезно да се разгледа един пълен сценарий на работа. Потребителят влиза в сайта и достига до началната страница. Оттам преминава към каталога, където използва търсене, филтриране и

сортиране, за да открие подходящ продукт. След това отваря детайли и решава да добави продукта в количката.

Ако потребителят не е логнат, системата го насочва към вход или регистрация. След удостоверяване операцията по добавяне в количка е разрешена. В количката потребителят може да промени количеството, да премахне продукт или да премине към checkout. На checkout страницата въвежда адрес и телефон, потвърждава наложен платеж и системата създава поръчка с всички нейни редове.

След успешна поръчка потребителят се пренасочва към секцията „Моите поръчки“, откъдето може да отвори детайлите на новосъздадената поръчка. Успоредно с това администраторът вижда същата поръчка в административния панел и може да проследи нейното съдържание. Така се затваря целият жизнен цикъл на покупката в рамките на приложението.

Този поток показва, че проектът „ЛапиХъб“ не е просто каталог с продукти, а завършена транзакционна система с потребителски акаунти, количка, поръчки и административно управление. Именно това го прави подходящ предмет на дипломна разработка.

Глава 5. Тестване на системата

Тестването е съществен етап от разработката на всяко приложение, защото чрез него се проверява дали реализираната функционалност отговаря на първоначалните изисквания и дали системата се държи коректно в различни ситуации. При e-commerce приложенията този процес е особено важен, понеже грешки в каталога, количката, поръчките или административните панели могат пряко да засегнат коректността на данните и работата на потребителите.

В проекта „ЛапиХъб“ тестването е проведено основно чрез функционално и ръчно тестване на потребителски сценарии. Това е подходящо за дипломна разработка, тъй като позволява преглед на цялостния работен поток – от началната страница до административното управление – и проверка дали всички основни модули взаимодействат правилно помежду си.

5.1. Цели на тестването

Основната цел на тестването е да се установи дали приложението работи коректно във всички ключови сценарии. Това включва проверка на публичната част, правилното поведение при регистрация и вход, работата на количката, коректното създаване на поръчки и ограниченията в административната част.

Допълнителна цел е да се валидира сигурността на системата – например дали обикновен потребител може да достъпи административни страници, дали потребител може да отвори чужда поръчка и дали защитените форми реагират правилно при некоректни данни.

Трети важен аспект е тестването на данните в базата. След изпълнение на дадено действие трябва да се провери дали в SQL Server се създават, променят или изтриват точно тези записи, които се очакват от бизнес логиката.

5.2. Тестова среда

Тестването на проекта се извършва в стандартна среда за разработка с .NET 8, SQL Server и ASP.NET Core MVC приложение, стартирано локално. Базата данни е създадена чрез миграции, а началните роли и данни са заредени чрез seed логика. Това гарантира, че тестовата среда съответства на реалната архитектура на системата и че резултатите от тестовете са релевантни за действителната работа на приложението.

Използването на локална тестова среда позволява директно наблюдение както на визуалния резултат в браузъра, така и на отражението в базата данни. Това е важно, защото при e-commerce системи не е достатъчно само да се види, че страницата изглежда правилно – необходимо е да се потвърди и че съответните записи действително са създадени или променени.

5.3. Тестване на публичната част

Първата група тестове е насочена към поведението на системата като публично достъпен сайт. Тук се проверява дали началната страница се отваря коректно, дали навигационната лента съдържа правилните връзки за гост и дали страницата с продукти визуализира seed-натия каталог.

Следващият важен сценарий е тестването на филтрите в каталога. Проверява се дали търсенето по име или марка връща правилни резултати, дали изборът на категория стеснява списъка коректно, дали филтрите по марка и ценови диапазон работят едновременно и дали сортирането по име и цена подрежда резултатите в очаквания ред.

Публичната страница с детайли за продукт също трябва да бъде проверена. Тук се валидира правилното визуализиране на снимка, марка, категория, цена и описание, както и показването на сходни продукти от същата категория. Освен това се проверява дали бутонът за добавяне в количката насочва нерегистрирания потребител към вход в системата.

Примерен чеклист за публичната част:

Тест	**Очакван резултат**
Отваряне на началната страница	Страницата се зарежда без грешка и показва carousel и секции за магазина.
Отваряне на каталога	Показва се списък с продукти с карти, снимки, цени и бутони.
Търсене по име/марка	Връщат се само съвпадащите продукти.
Филтър по категория	Визуализират се само продукти от избраната категория.
Сортиране по цена	Продуктите се подреждат възходящо или низходящо.
Отваряне на детайли	Показва се пълна страница за продукта.

Добавяне в количка като гост	Потребителят се пренасочва към страницата за вход.

5.4. Тестване на регистрация, вход и роли

Втората група тестове е свързана с идентификацията на потребителите. Проверява се дали регистрационната форма изисква всички задължителни полета – име, фамилия, телефон, имейл, парола и потвърждение на парола – и дали валидиращите съобщения са изписани на български език.

След успешна регистрация трябва да се потвърди, че в базата данни е създаден нов запис в AspNetUsers и че потребителят е добавен към роля User чрез AspNetUserRoles. При тест за вход се проверява както успешният сценарий, така и отрицателният – въвеждане на грешен имейл или парола трябва да доведе до показване на съобщението „Грешен имейл или парола.“.

Ролевото поведение също трябва да бъде валидирано. Гостът не трябва да вижда табло или административен dropdown. Логнатият потребител трябва да вижда таблото и поздрав по име. Администраторът трябва да вижда допълнително меню „Администриране“ и да може да отваря административните страници.

5.5. Тестване на количка и checkout

Количката е модул с множество малки, но важни операции, затова трябва да се тества подробно. Първо се проверява дали добавянето на продукт създава CartItem запис за текущия потребител. След това се валидира поведението при повторно добавяне на същия продукт – вместо нов ред трябва да се увеличи количеството на съществуващия ред.

Промяната на количеството чрез dropdown се тества за различни стойности между 1 и 10. Проверява се дали контролерът записва новото количество и дали общата стойност на реда и количката се обновяват правилно. Отделно се проверява дали продуктът може да бъде премахнат от количката и дали списъкът остава коректен след това.

Checkout процесът се тества като завършен транзакционен поток. Първо се проверява дали при празна количка потребителят не може да отвори checkout формата. След това се валидира успешен сценарий, при който количката съдържа продукти, формата се попълва с адрес и телефон, създава се поръчка, записват се

редовете в OrderItems и количката се изчиства. Накрая се проверява дали новата поръчка се вижда в „Моите поръчки“.

Чеклист за количка и checkout:

Тест	**Очакван резултат**
Добавяне на нов продукт в количка	Създава се CartItem с количество 1.
Повторно добавяне на същия продукт	Количеството се увеличава, не се създава нов ред.
Промяна на количество	Записва се новата стойност и сумите се обновяват.
Премахване на продукт	Редът се изтрива от CartItems.
Checkout с празна количка	Потребителят се връща към количката с предупреждение.
Успешен checkout	Създават се Order и OrderItems, а количката се изчиства.

5.6. Тестване на потребителските поръчки

След създаване на поръчка се проверява дали списъкът „Моите поръчки“ визуализира коректно номер на поръчка, дата, статус, брой артикули и обща сума. Следва да се отвори детайлната страница и да се потвърди, че тя съдържа правилен адрес, телефон, метод на плащане, всички продукти в поръчката и точни междинни и крайни суми.

Особено важен е тестът за ограничен достъп. При ръчна подмяна на идентификатора в URL адреса с поръчка на друг потребител системата не трябва да показва чужди данни. В проекта това се реализира чрез филтриране по UserId в контролера и се проверява именно чрез подобен отрицателен сценарий.

5.7. Тестване на административната част

Административната част се тества поотделно за всеки модул. При категориите се проверява дали CRUD операциите работят коректно и дали изтриването е блокирано, когато съществуват свързани продукти. При продуктите се проверява създаване, редактиране, визуализация и поведение при изтриване на продукт, участващ в количка или поръчка.

При поръчките се валидира дали административният списък показва всички заявки, независимо от потребителя, и дали детайлите съдържат клиента, адреса, телефона, продуктите и сумите. При потребителите се проверява списъкът с роли, броя поръчки и артикули в количката, както и правилното поведение при редакция и изтриване.

Най-важните отрицателни административни сценарии включват: блокиране на self-delete за администратора, блокиране на изтриване на потребител с поръчки, блокиране на изтриване на категория с продукти и правилно съобщение при невъзможност за изтриване на продукт, участващ в поръчка.

5.8. Обобщение на резултатите от тестването

Проведеното тестване показва, че системата покрива основните си функционални изисквания. Публичната част работи стабилно, каталогът позволява ефективно намиране на продукти, а потребителските сценарии за количка и поръчка завършват успешно и коректно записват данни в базата.

Ролевата логика и ограниченията за достъп също функционират според очакванията. Потребителят не вижда административни панели, а чуждите поръчки остават недостъпни. Администраторът разполага с централизирано управление и защитни механизми срещу опасни операции.

Следователно може да се заключи, че приложението „ЛапиХъб“ е реализирано като завършена и работеща e-commerce система, която е годна за демонстрация, оценяване и бъдещо разширяване.

ЗАКЛЮЧЕНИЕ

Разработката на „ЛапиХъб“ демонстрира създаването на завършено уеб приложение за електронна търговия, реализирано със съвременни .NET технологии и организирано по утвърдени архитектурни принципи. Проектът обединява публичен продуктов каталог, потребителска част с количка и поръчки и административен панел за управление на съдържанието и потребителите.

В хода на реализацията са приложени важни знания и умения от няколко области: уеб програмиране, обектно моделиране, работа с релационни бази данни, сигурност на уеб приложения, потребителски интерфейс и софтуерна архитектура. Използването на ASP.NET Core MVC, Entity Framework Core, SQL Server и ASP.NET Identity дава надеждна основа, върху която системата е изградена последователно и логически.

Постигнатите резултати показват, че поставената цел е изпълнена. Системата позволява на госта да разглежда каталога, на регистрирания потребител – да създава поръчки и да следи тяхната история, а на администратора – да управлява основните данни в магазина. Реализирани са и редица защитни механизми, които пазят приложението от невалидни действия и нарушаване на целостта на данните.

Освен като учебен проект, „ЛапиХъб“ може да се разглежда и като добра основа за бъдещо надграждане. Неговата архитектура позволява добавяне на нови функционалности и адаптиране към други продуктови тематики. По този начин разработката изпълнява успешно както практическа, така и образователна роля и представлява завършена дипломна реализация на e-commerce система.

ИЗПОЛЗВАНА ЛИТЕРАТУРА

1. Колисниченко Денис, Преводач: Влади Владев (2017), HTML 5 & CSS 3 - практическо програмиране за начинаещи, Асеневци; София.
2. Манев Красимир, Андонов Филип, Манева Нели (2021), Информатика за профилирана подготовка 12. клас, модул "Релационен модел на бази от данни", Изкуства, София.
3. Наков Светлин и колектив, „Основи на програмирането“ (със C#) Българска асоциация на разработчиците на софтуер, София, (2017)
4. Наков Светлин – GitHub

https://github.com/BG-IT-Edu/School-Programming/tree/main/Courses/Applied-Programmer?fbclid=IwAR2Oh_EMlKNRSXbHuSCiX1ay0wjPbwK2EKHHA8IdT8n1BxKRtrUNp0uyqaU *(към 03.04.2026)*

5. Alls Jason (Юли 17, 2020), Clean Code in C#: Refactor your legacy C# code base and improve application performance by applying best practices, Packt Publishing; Бирмингам, Великобритания;
6. Freeman Adam (2021), Pro ASP.NET Core Identity: Under the Hood with Authentication and Authorization in ASP.NET Core 5 and 6 Applications, Apress; Ню Йорк, САЩ.
7. Hanselman Scott https://www.hanselman.com/blog/ *(към 03.04.2026)*
8. Bootstrap – Официален сайт https://getbootstrap.com/ *(към 28.03.2026)*
9. C# Programming Guide – Microsoft Docs

https://docs.microsoft.com/en-us/dotnet/csharp/*(към 28.03.2026)*

10. ChatGPT – OpenAI https://openai.com/chatgpt *(към 03.04.2026)*
11. Microsoft Learn – ASP.NET Core Tutorials https://learn.microsoft.com/en-us/training/paths/build-web-applications-with-aspnet-core/ *(към 03.04.2026)*
12. Stack Overflow https://stackoverflow.com/ *(към 07.04.2026)*

Ръководство за потребителя

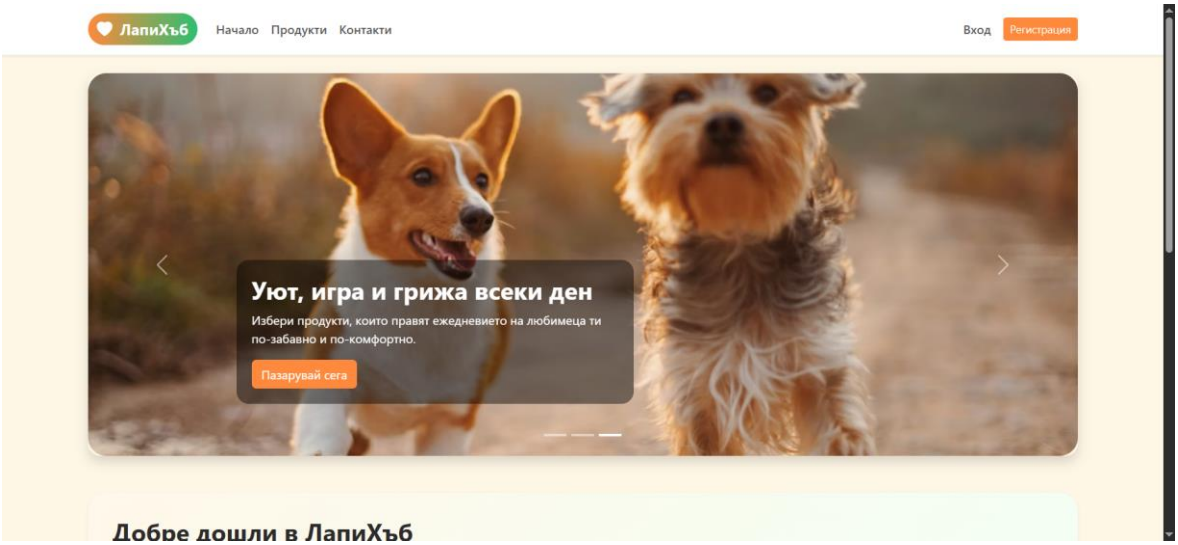
Настоящата глава описва работата със системата от гледна точка на крайния потребител и администратора. Целта е всеки тип потребител да може лесно да разбере какви стъпки трябва да изпълни, за да използва приложението според своите права. Ръководството е структурирано по роли, защото интерфейсът и достъпните действия в „ЛапиХъб“ се променят според това дали потребителят е гост, логнат клиент или администратор.

Въпреки че приложението е интуитивно, наличието на кратко и ясно потребителско ръководство е важно както за дипломната документация, така и за практическото използване на системата. То служи и като доказателство, че проектът е разработен с мисъл за реална употреба, а не само като демонстрация на код.

Работа със системата като гост

Гост е всеки посетител на сайта, който не е влязъл в системата. При първо отваряне на приложението той вижда началната страница на „ЛапиХъб“, където може да се запознае с темата на магазина и да премине към продуктовия каталог.

От страницата „Продукти“ гостът може да разглежда каталога, да използва търсене по име или марка, да филтрира по категория, марка или цена и да сортира резултатите. Чрез бутона „Детайли“ може да отвори пълната страница на конкретен продукт и да прегледа неговото описание.



Ако гостът натисне бутон за добавяне в количката, системата изисква вход в профил. Това е нормално и очаквано поведение, тъй като количката в системата е свързана с конкретен удостоверен потребител.

Последователността от стъпки за гост е следната:

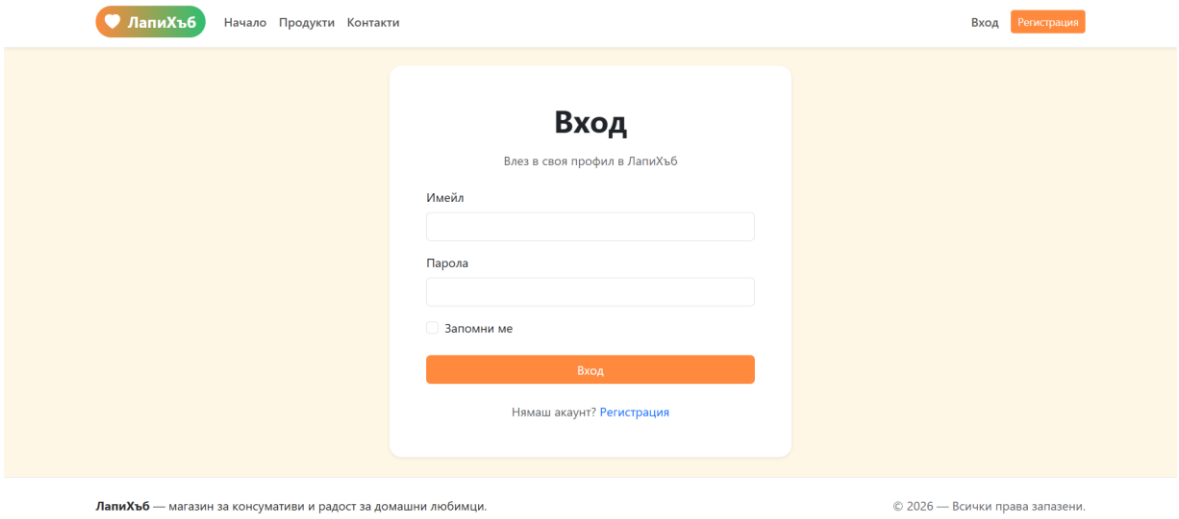
1. Отваряне на началната страница на магазина.
2. Преминаване към меню „Продукти“.
3. Използване на търсене и филтри за намиране на желан артикул.
4. Отваряне на страница с детайли при интерес към конкретен продукт.
5. Регистрация или вход при желание за покупка.

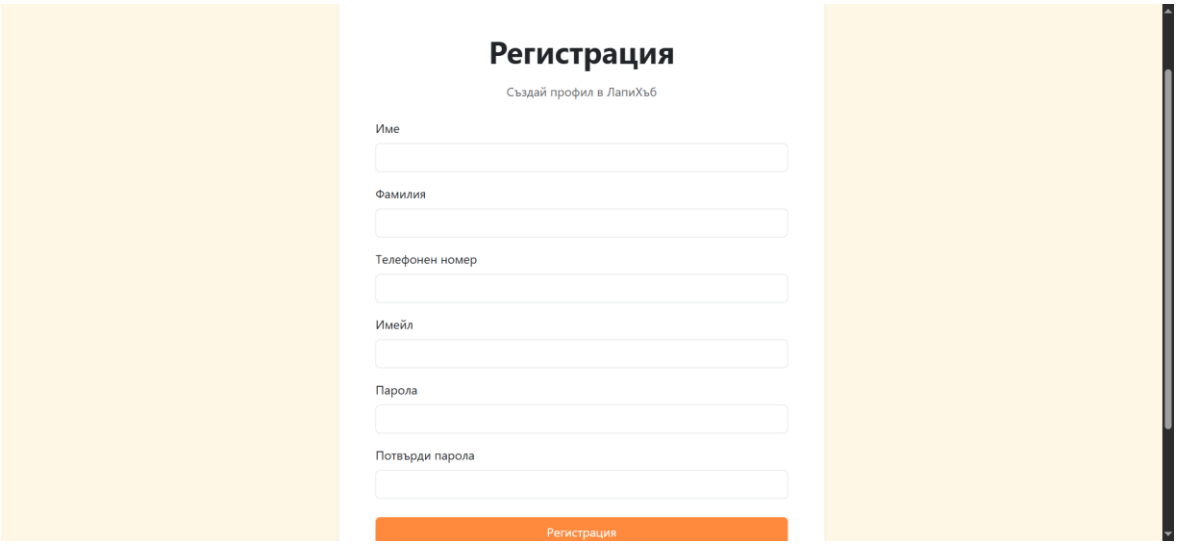
Регистрация и вход

За да използва количката и поръчките, посетителят трябва да има профил в системата. Регистрацията се извършва чрез страницата „Регистрация“, където се попълват собствено име, фамилия, телефонен номер, имейл, парола и потвърждение на паролата. След успешна регистрация системата автоматично създава потребителски акаунт и присвоява роля User.

След регистрация потребителят може да влиза в системата чрез страницата „Вход“. Необходимо е да въведе имейл и парола. При грешни данни се показва съобщение за неуспешен вход, а при коректни – системата го допуска до личното табло и допълнителните функции на магазина.

След успешен вход навигацията се променя. В горната част на екрана се появява поздрав по име, връзка към таблото и възможност за изход от системата.

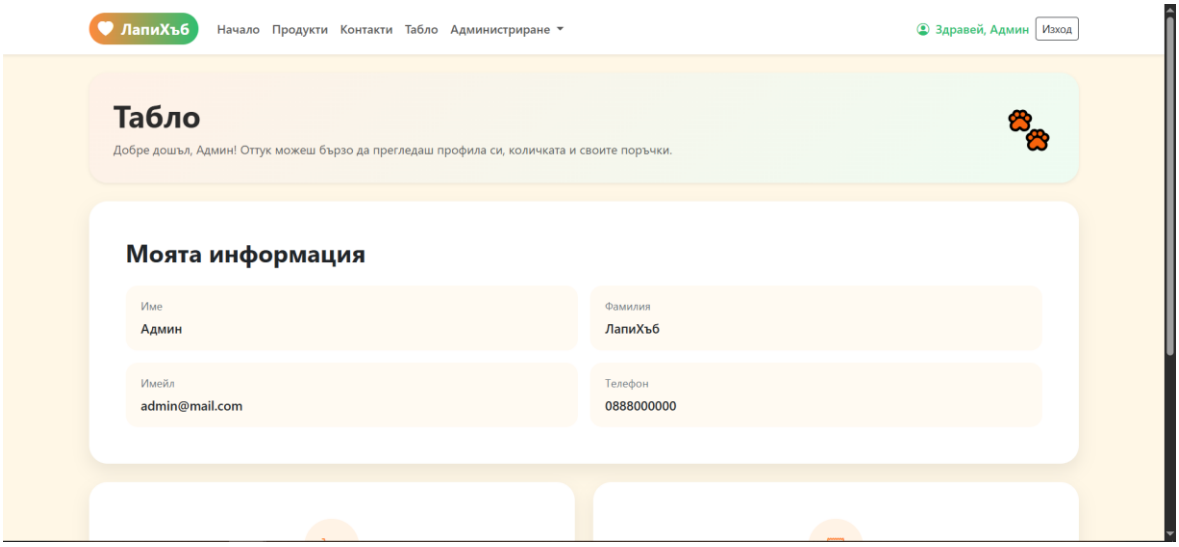




Работа с таблото на потребителя

След вход потребителят получава достъп до страница „Табло“. Тази страница служи като кратък преглед на профила и съдържа личните данни – име, фамилия, имейл и телефон. Освен това в нея са разположени бутони за бърз достъп до количката и до историята на поръчките.

Таблото има за цел да улесни потребителя и да обедини най-често използваните действия. Вместо да търси отделните функции в менюто, той може директно от таблото да отвори своята количка или своите поръчки.



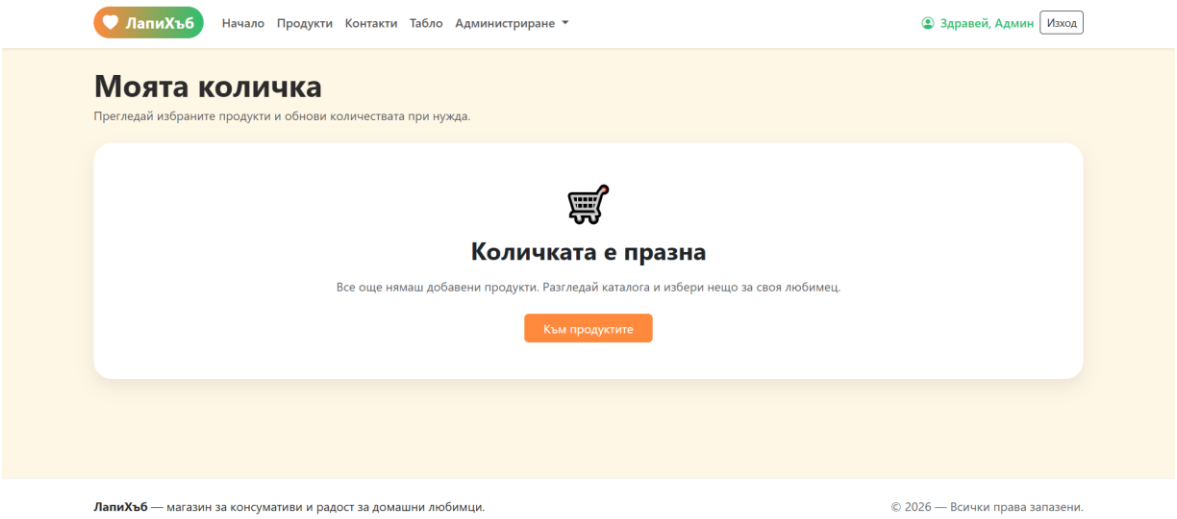
Работа с количката

Количката съдържа всички продукти, които потребителят е избрал, но все още не е поръчал. След натискане на бутон „Добави в количката“ за даден продукт

той се записва в количката с начално количество 1. Ако същият продукт бъде добавен отново, количеството му се увеличава.

В страницата на количката потребителят вижда списък с всички избрани продукти, тяхната снимка, име, марка, единична цена, количество и обща стойност за реда. В дясната част на екрана се визуализира обобщение с крайната стойност на количката.

Количеството на всеки продукт може да се променя чрез падащо меню със стойности от 1 до 10. Ако потребителят реши, че даден продукт не е необходим, той може да го премахне чрез бутона за изтриване. След това системата обновява общата сума.

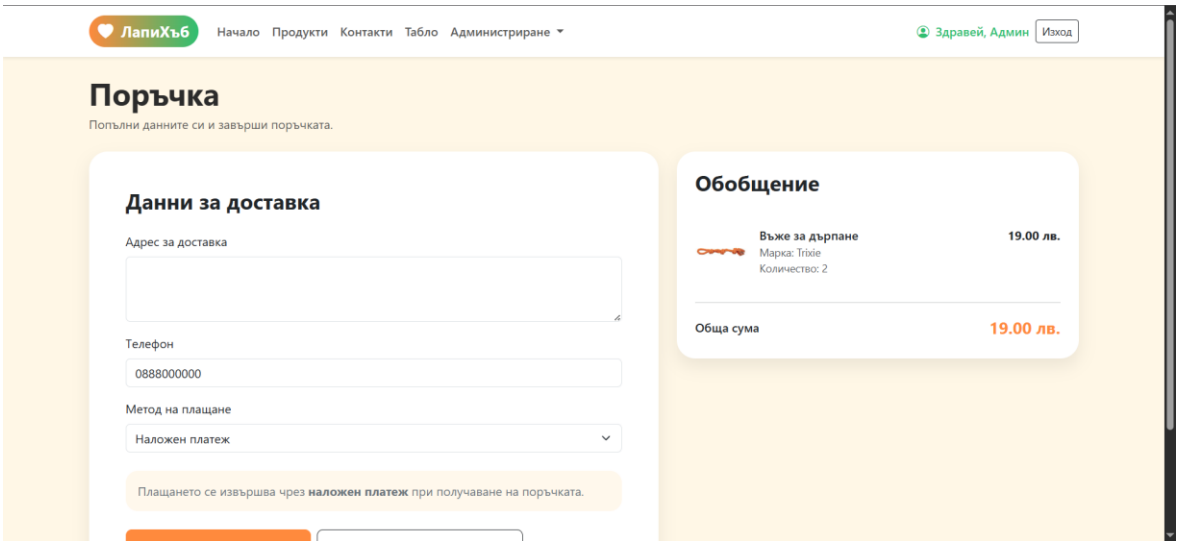


Създаване на поръчка

След като количката съдържа желаните продукти, потребителят натиска бутона „Поръчай“. Системата го отвежда към checkout страницата, където се попълват необходимите данни за доставка. Формата включва адрес, телефон и начин на плащане. В рамките на проекта е реализиран наложен платеж като опростен начин за плащане.

Под формата се визуализира обобщение на продуктите в количката, включително имената, количествата и общата стойност на поръчката. Това позволява на потребителя да прегледа окончателно своя избор преди потвърждаване.

След натискане на „Потвърди поръчката“ системата създава поръчка, записва нейните елементи в базата данни и изчиства количката. Потребителят се пренасочва към страницата „Моите поръчки“, където вижда новосъздадения запис.

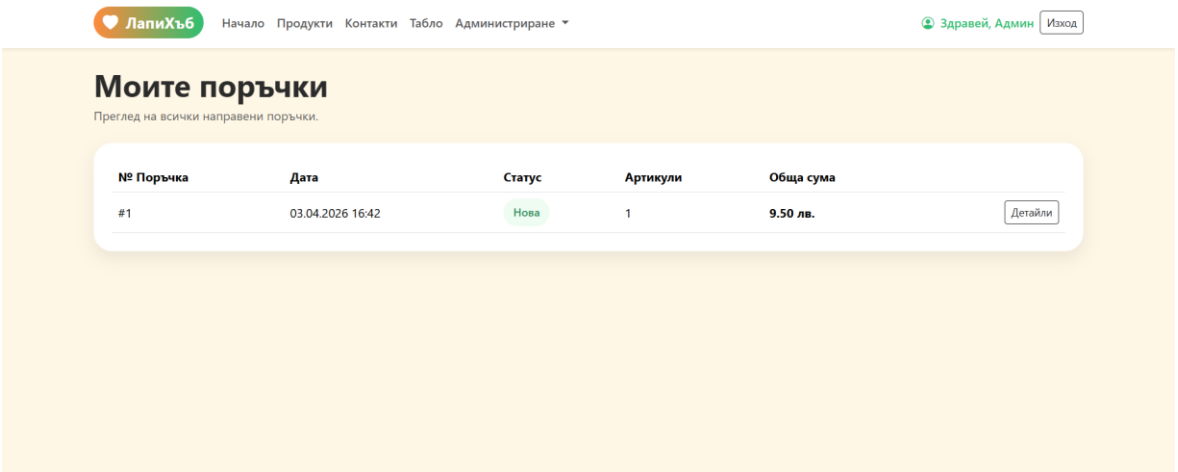


Преглед на поръчките

Страницата „Моите поръчки“ съдържа история на всички поръчки, създадени от текущия потребител. За всяка поръчка се показват номер, дата, статус, брой артикули и обща сума. Чрез бутона „Детайли“ може да се отвори подробен изглед на поръчката.

Детайлната страница показва адреса, телефона, начина на плащане, статуса и точния списък на продуктите в поръчката. Това позволява на потребителя да проследява какво е поръчал и да провери дали данните са правилно записани.

Важно е да се отбележи, че системата позволява преглед само на собствени поръчки. Ако потребителят се опита да отвори чужда поръчка чрез ръчна промяна на адреса, достъпът се отказва.

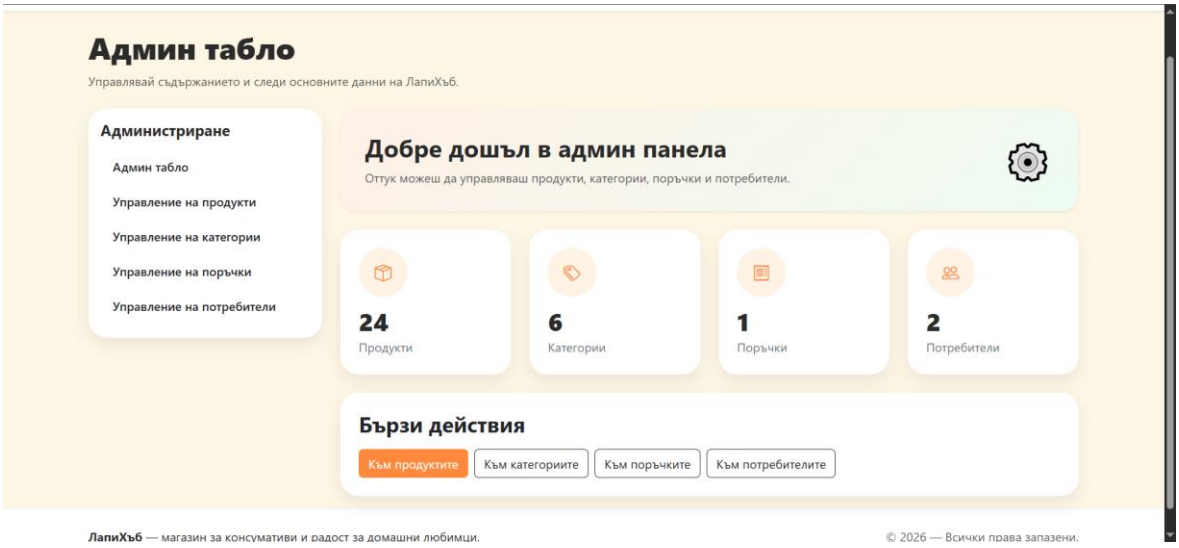


Работа със системата като администратор

Администраторът влиза в системата по същия начин като обикновения потребител, но след успешен вход разполага с допълнително меню „Администриране“. От него може да отвори администраторското табло и отделните панели за продукти, категории, поръчки и потребители.

Админ таблото показва общи статистики за броя продукти, категории, поръчки и потребители. Това е отправна точка към всички останали административни действия.

В панела за категории администраторът може да създава нови категории, да ги редактира и да ги изтрива, ако няма свързани продукти. В панела за продукти може да управлява пълния продуктов каталог, включително снимки, описания и цени. В панела за поръчки може да разглежда всички направени заявки, а в панела за потребители – да преглежда и редактира данните на регистрираните акаунти.



Практически насоки за работа

За най-добро използване на системата е препоръчително потребителят първо да се запознае с филтрите в каталога, а след това да използва таблото като централен достъп до своите функции. Ако системата се използва за демонстрационни цели, е удобно да се създадат няколко тестови акаунта, чрез които да се провери разделението на ролите и историята на поръчките.

За администратора е препоръчително първо да провери категориите и продуктите, след което да тества създаване на поръчка с потребителски акаунт и да

проследи как тя се появява в административния панел. По този начин може да се валидира целият работен поток от край до край.

Инсталация, пренасяне и поддръжка на проекта

За да бъде дипломният проект практически полезен, той трябва не само да работи на машината на разработчика, но и да може да бъде пренесен и стартиран на друга среда. Поради това настоящата глава описва необходимите условия за инсталация, създаване на база данни, изпълнение на миграции, seed на начални данни и типични проблеми, които могат да възникнат.

Тъй като „ЛапиХъб“ е разработен с утвърдени .NET технологии, процесът по стартиране е сравнително ясен, стига да са налични нужните инструменти и правилно конфигурирана връзка към SQL Server.

Необходими условия за стартиране

За стартиране на проекта са необходими инсталирани .NET 8 SDK, среда за разработка като Visual Studio 2022 или друга съвместима IDE, както и работещ екземпляр на SQL Server или LocalDB. Необходимо е също проектът да съдържа всички NuGet зависимости, свързани с ASP.NET Core MVC, Entity Framework Core и Identity.

Конфигурацията на връзката към базата данни се намира в appsettings.json под ключ DefaultConnection. При пренасяне на проекта на друга машина тази стойност може да бъде коригирана според наличния SQL Server инстанс.

Създаване на базата данни

След отваряне на проекта първата стъпка е изпълнение на миграциите. Това може да стане чрез Package Manager Console или чрез командния ред. След създаване на базата данни и при първо стартиране на приложението се изпълнява seed логиката, която създава ролите и административния акаунт.

Типичната последователност е: възстановяване на NuGet пакетите, компилиране на проекта, изпълнение на Add-Migration при нужда от нови промени и Update-Database за прилагане на миграциите. При вече готов проект е достатъчно Update-Database да създаде нужната база на новата машина.

Примерни команди за работа с миграции:

```
Add-Migration InitialCreate
Update-Database
```

Seed на роли и администратор

След стартиране на приложението Program.cs извиква seed логиката, която създава ролите Admin и User и административен акаунт по подразбиране. Това е важно за първоначалния вход в системата, особено когато проектът се демонстрира пред преподавател или се прехвърля в нова среда.

Началният администратор може да бъде използван за вход в административния панел и за проверка на всички CRUD функционалности. В реална среда тези начални данни следва да бъдат променени, но за целите на дипломния проект те служат като удобен механизъм за първоначална конфигурация.

Често срещани проблеми и решения

Един от често срещаните проблеми е липсата на роли в базата данни. Ако таблицата AspNetRoles е празна и администраторското меню не се появява, трябва да се провери дали seed логиката се изпълнява коректно при стартиране на приложението.

Друг възможен проблем е неуспешно пренасочване към вход при защитени действия. В проекта за вход се използват стандартните Identity страници от зоната /Identity/Account/Login, поради което е важно приложението да има MapRazorPages() в Program.cs. Ако тази настройка липсва, страниците за логин и регистрация няма да функционират коректно.

Възможно е да възникнат и проблеми със стари миграции, особено ако моделите са били променяни многократно по време на разработката. В подобна ситуация е необходима внимателна синхронизация между модела, миграциите и реалната база данни.

При тестване на административното изтриване на продукти и категории могат да се появят ограничения, свързани с релациите в базата данни. Това поведение е очаквано и е част от защитата на системата срещу неконсистентни операции.

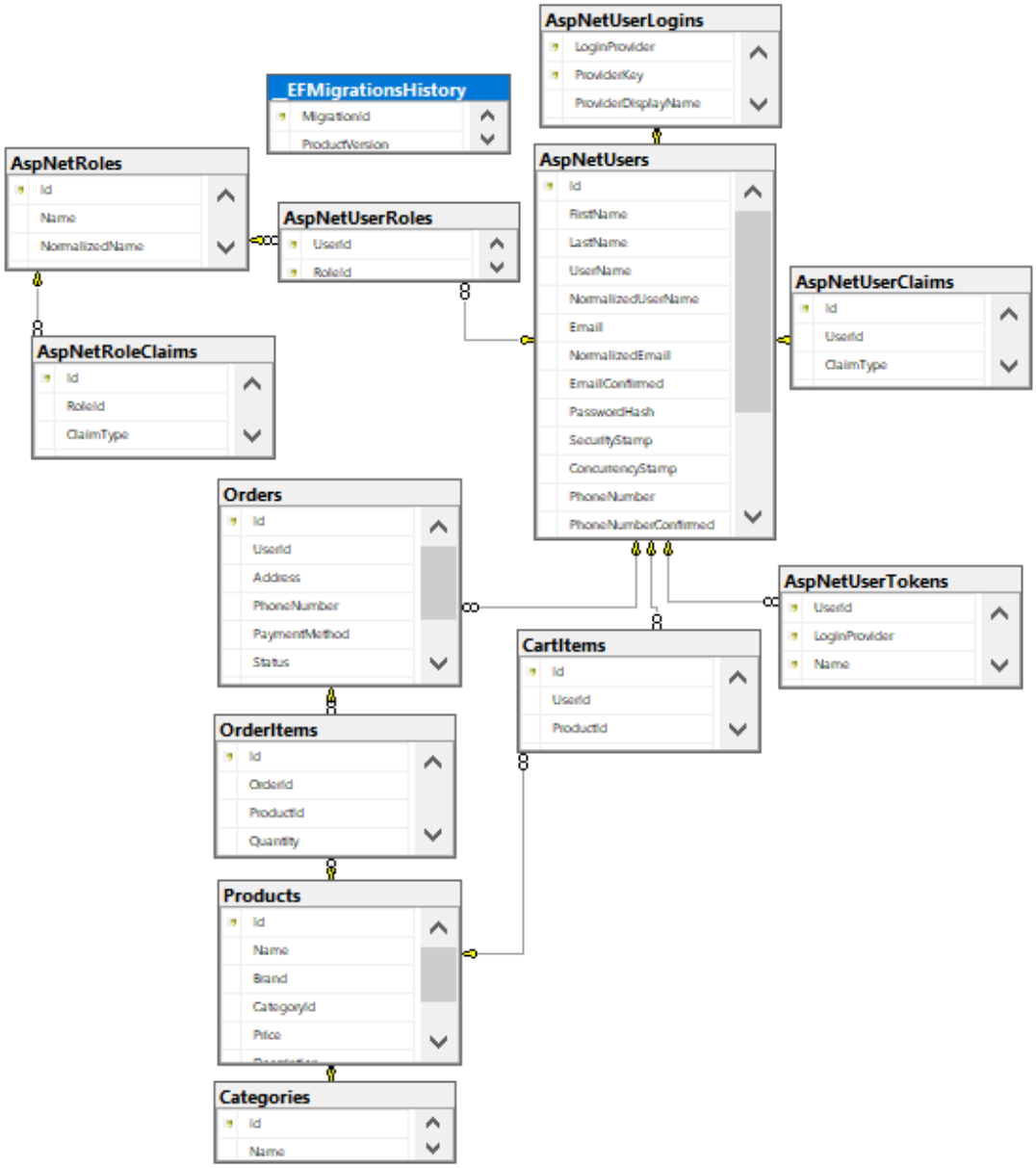
Поддръжка и възможности за бъдещо разширяване

Проектът е изграден така, че да позволява относително лесно разширяване. В бъдеще могат да бъдат добавени нови категории продукти, по-сложна логика за статуси на поръчките, търсене с повече критерии, клиентски профили с адреси, реална интеграция с платежни системи и по-сложно управление на наличности.

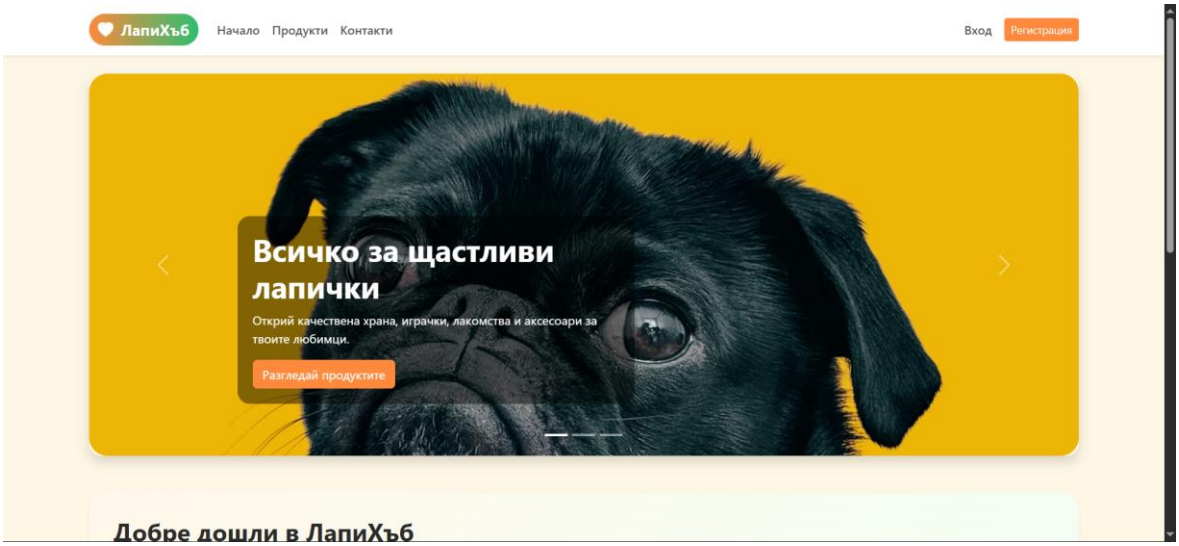
В архитектурно отношение използването на MVC, view model-и и Entity Framework Core означава, че новите функционалности могат да бъдат добавяни по модулен начин. Това е едно от най-големите предимства на избрания подход за разработка.

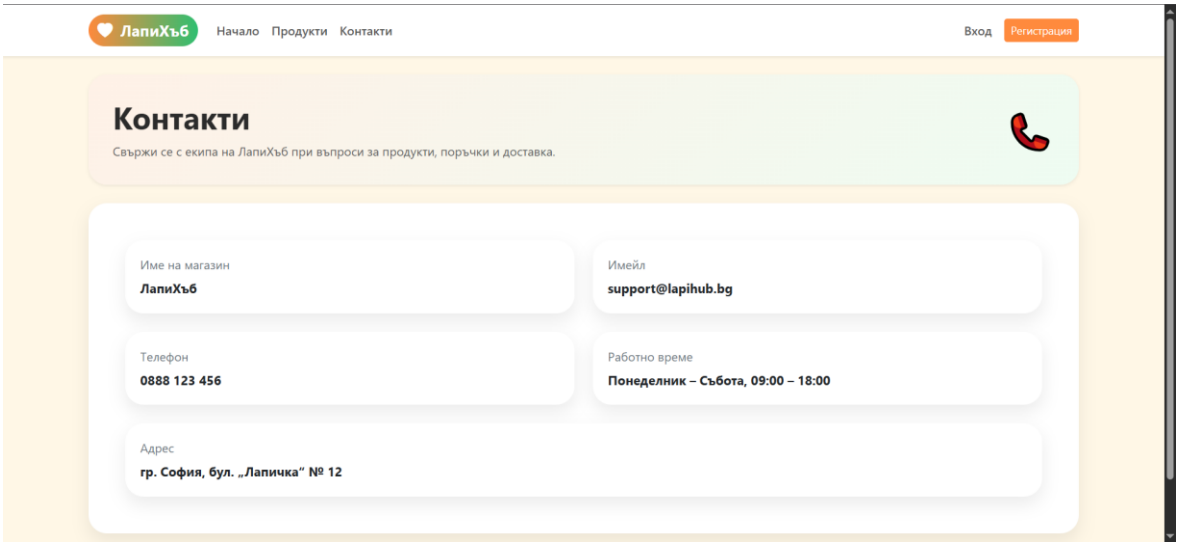
ПРИЛОЖЕНИЯ

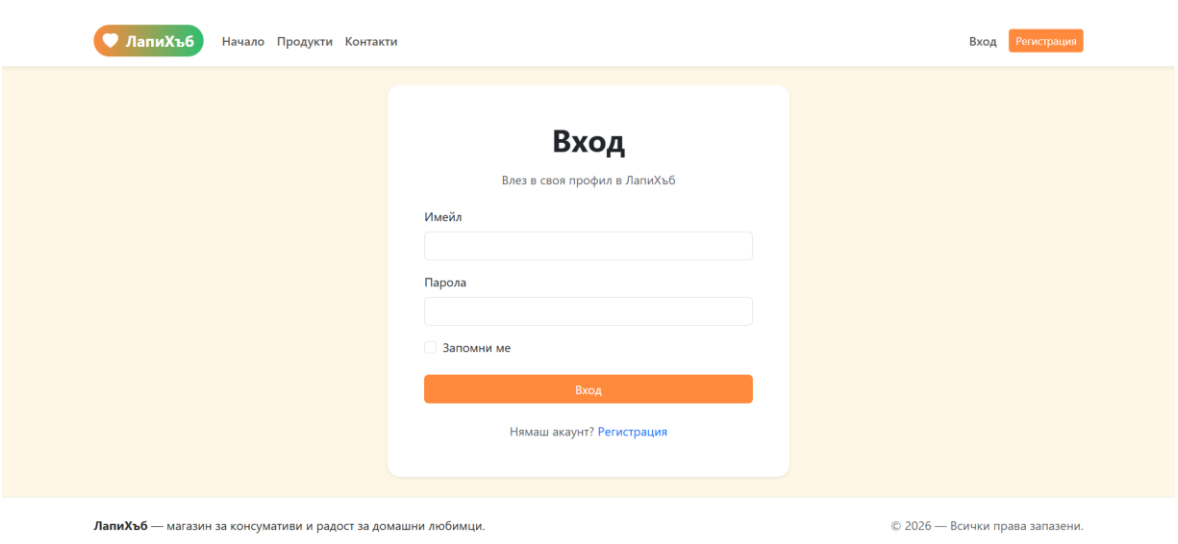
Приложение 1: **ER диаграма на базата данни на ДелфинШоп (таблици и връзки).**



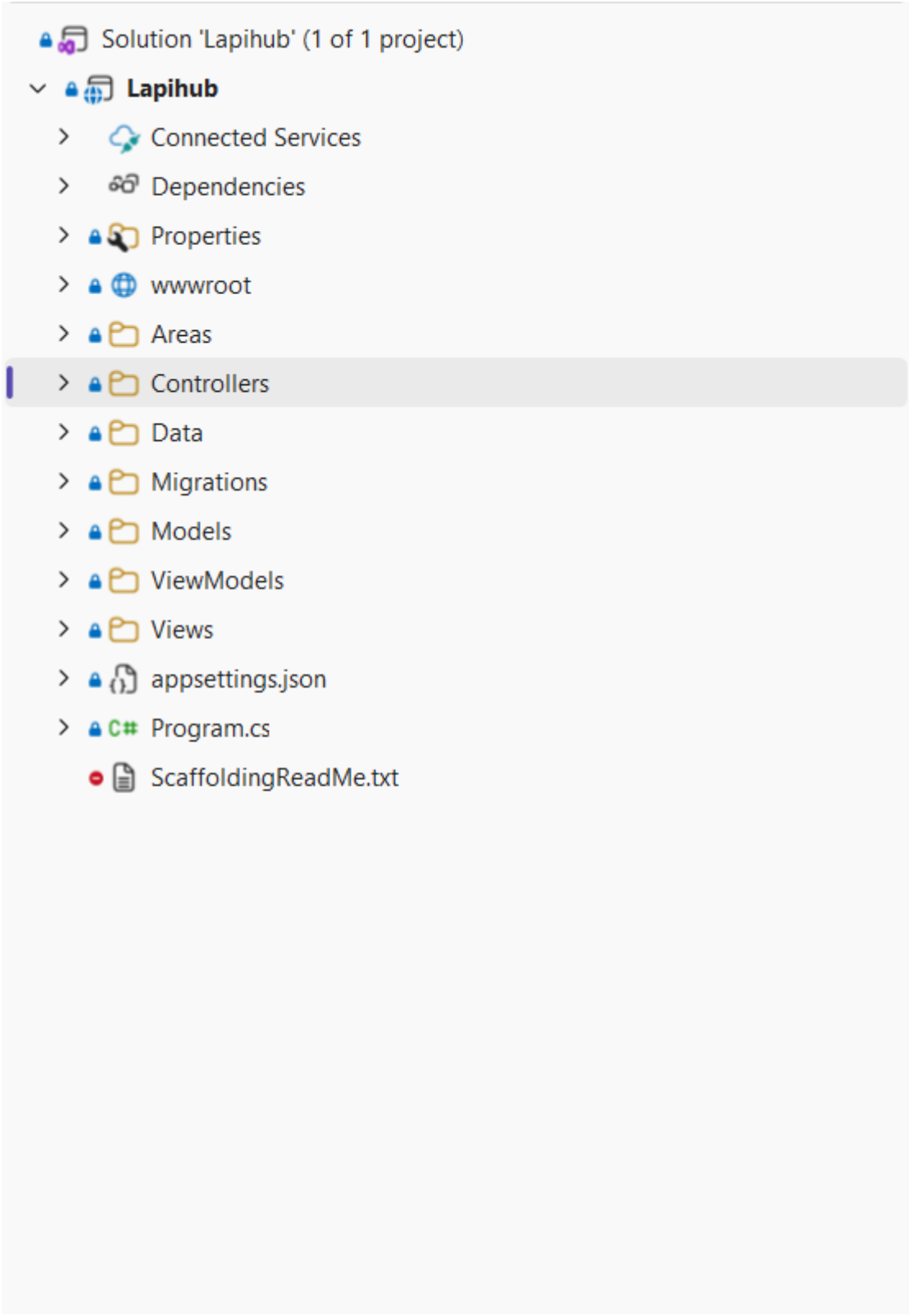
Приложение 2: Снимки на избрани екрани от приложението ДелфинШоп (начална страница, каталог, детайл на продукт, количка, форма за поръчка, административни панели).



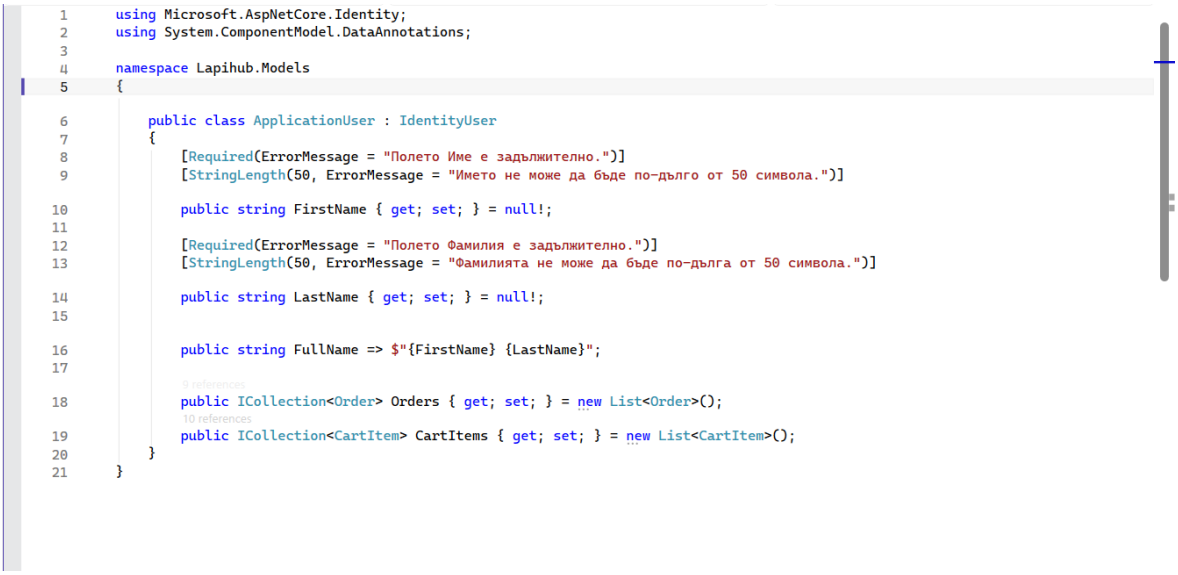




Приложение 3: Архитектура на приложението



Приложение 4: ApplicationUser



```
using Microsoft.AspNetCore.Identity;
using System.ComponentModel.DataAnnotations;

namespace Lapihub.Models
{

    public class ApplicationUser : IdentityUser
    {
        [Required(ErrorMessage = "Полето Име е задължително.")]
        [StringLength(50, ErrorMessage = "Името не може да бъде по-дълго от 50 символа.")]

        public string FirstName { get; set; } = null!;

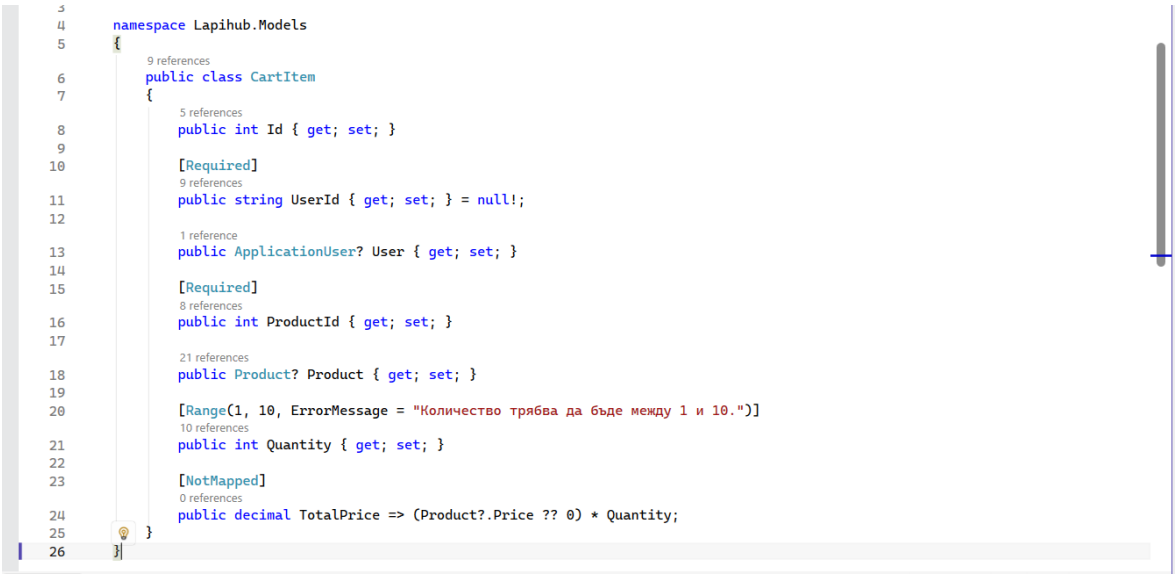
        [Required(ErrorMessage = "Полето Фамилия е задължително.")]
        [StringLength(50, ErrorMessage = "Фамилията не може да бъде по-дълга от 50 символа.")]

        public string LastName { get; set; } = null!;


        public string FullName => $"{FirstName} {LastName}";

        9 references
        public ICollection<Order> Orders { get; set; } = new List<Order>();
        10 references
        public ICollection<CartItem> CartItems { get; set; } = new List<CartItem>();
    }
}
```

Приложение 5: CartItem.cs



```
namespace Lapihub.Models
{
    9 references
    public class CartItem
    {
        5 references
        public int Id { get; set; }

        [Required]
        9 references
        public string UserId { get; set; } = null!;

        1 reference
        public ApplicationUser? User { get; set; }

        [Required]
        8 references
        public int ProductId { get; set; }

        21 references
        public Product? Product { get; set; }

        [Range(1, 10, ErrorMessage = "Количество трябва да бъде между 1 и 10.")]
        10 references
        public int Quantity { get; set; }

        [NotMapped]
        0 references
        public decimal TotalPrice => (Product?.Price ?? 0) * Quantity;
    }
}
```

Приложение 6: Category.cs



```
using System.ComponentModel.DataAnnotations;

namespace Lapihub.Models
{
    10 references
    public class Category
    {
        18 references
        public int Id { get; set; }

        [Required(ErrorMessage = "Името на категорията е задължително.")]
        [StringLength(100, ErrorMessage = "Името на категорията не може да бъде по-дълго от 100 символа.")]
        25 references
        public string Name { get; set; } = null!;

        9 references
        public ICollection<Product> Products { get; set; } = new List<Product>();
    }
}
```

Приложение 7: Order.cs

```
public class Order
{
    9 references
    public int Id { get; set; }

    [Required]
    4 references
    public string UserId { get; set; } = null!;

    13 references
    public ApplicationUser? User { get; set; }

    [Required(ErrorMessage = "Адресът е задължителен.")]
    [StringLength(250, ErrorMessage = "Адресът не може да бъде по-дълъг от 250 символа.")]
    3 references
    public string Address { get; set; } = null!;

    [Required(ErrorMessage = "Телефонът е задължителен.")]
    [Phone(ErrorMessage = "Моля, въведете валиден телефонен номер.")]
    3 references
    public string PhoneNumber { get; set; } = null!;

    [Required(ErrorMessage = "Методът на плащане е задължителен.")]
    5 references
    public PaymentMethod PaymentMethod { get; set; }

    6 references
    public OrderStatus Status { get; set; } = OrderStatus.Нова;

    [Column(TypeName = "decimal(18,2)")]
    7 references
    public decimal TotalAmount { get; set; }

    8 references
    public DateTime CreatedOn { get; set; } = DateTime.Now;

    14 references
    public ICollection<OrderItem> OrderItems { get; set; } = new List<OrderItem>();
}
```

Приложение 8: OrderItem.cs

```
using System.ComponentModel.DataAnnotations.Schema;

namespace Lapihub.Models
{
    public class OrderItem
    {
        public int Id { get; set; }

        public int OrderId { get; set; }

        1 reference
        public Order? Order { get; set; }

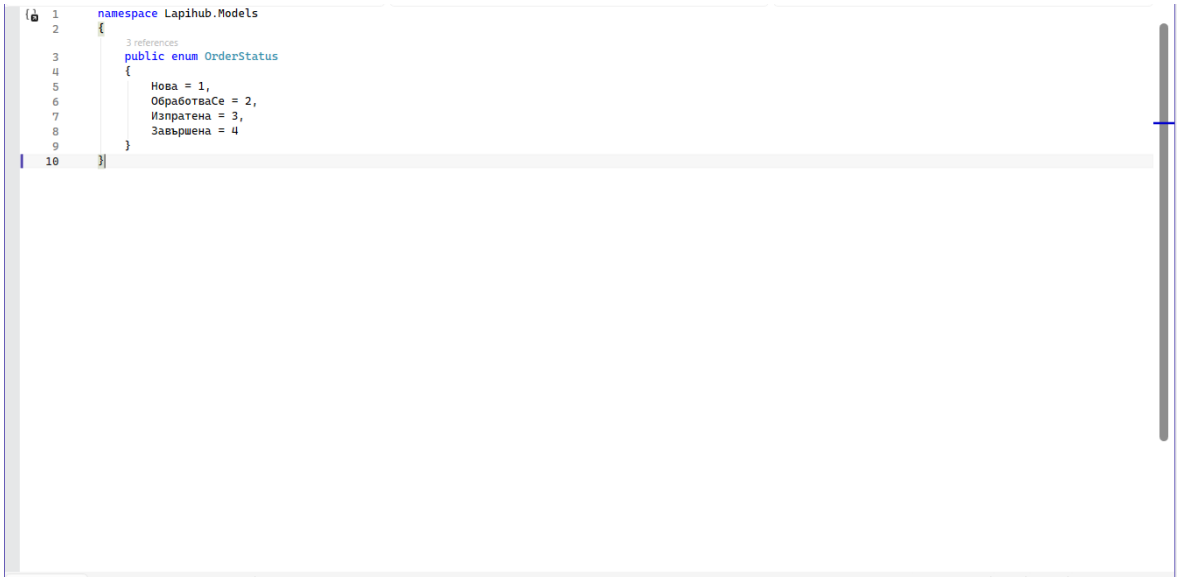
        4 references
        public int ProductId { get; set; }

        9 references
        public Product? Product { get; set; }

        3 references
        public int Quantity { get; set; }

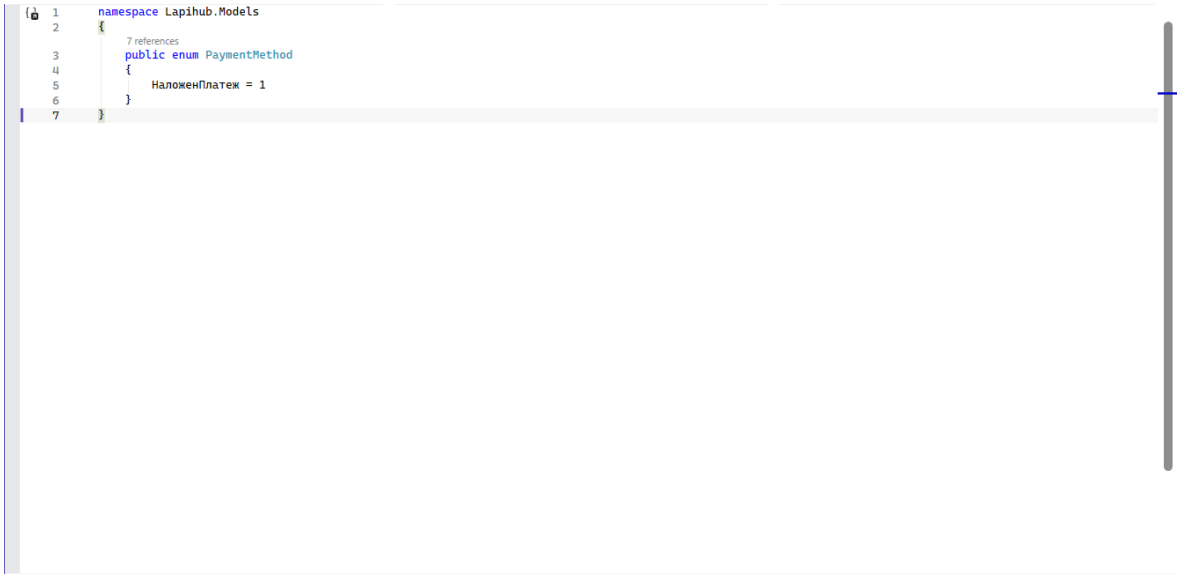
        [Column(TypeName = "decimal(18,2)")]
        4 references
        public decimal Price { get; set; }
    }
}
```

Приложение 9: OrderStatus.cs



```
namespace Lapihub.Models
{
    3 references
    public enum OrderStatus
    {
        Нова = 1,
        ОбработваСе = 2,
        Изпратена = 3,
        Завършена = 4
    }
}
```

Приложение 10: PaymentMethod.cs



```
namespace Lapihub.Models
{
    7 references
    public enum PaymentMethod
    {
        НаложенПлатеж = 1
    }
}
```

Приложение 11: Product.cs

```
namespace Lapihub.Models
{
    38 references
    public class Product
    {
        37 references
        public int Id { get; set; }

        [Required(ErrorMessage = "Името на продукта е задължително.")]
        [StringLength(150, ErrorMessage = "Името на продукта не може да бъде по-дълго от 150 символа.")]
        49 references
        public string Name { get; set; } = null!;

        [Required(ErrorMessage = "Марката е задължителна.")]
        [StringLength(100, ErrorMessage = "Марката не може да бъде по-дълга от 100 символа.")]
        42 references
        public string Brand { get; set; } = null!;

        [Required(ErrorMessage = "Категорията е задължителна.")]
        32 references
        public int CategoryId { get; set; }

        13 references
        public Category? Category { get; set; }

        [Required(ErrorMessage = "Цената е задължителна.")]
        [Column(TypeName = "decimal(18,2)")]
        [Range(0.01, 100000, ErrorMessage = "Цената трябва да бъде между 0.01 и 100000.")]
        44 references
        public decimal Price { get; set; }

        [Required(ErrorMessage = "Описанието е задължително.")]
        [StringLength(2000, ErrorMessage = "Описанието не може да бъде по-дълго от 2000 символа.")]
        31 references
        public string Description { get; set; } = null!;

        [Required(ErrorMessage = "Снимката е задължителна.")]
        [Display(Name = "Снимка")]
        38 references
        public string ImageUrl { get; set; } = null!;

        1 reference
        public ICollection<CartItem> CartItems { get; set; } = new List<CartItem>();
        1 reference
        public ICollection<OrderItem> OrderItems { get; set; } = new List<OrderItem>();
    }
}
```

Приложение 12: _Layout.cshtml

```
    @RenderSection("Styles", required: false)
</head>
<body>
    <header>
        <nav class="navbar navbar-expand-lg shadow-sm">
            <div class="container">
                <a class="navbar-brand d-flex align-items-center gap-2" asp-controller="Home" asp-action="Index">
                    <span class="brand-badge">
                        <i class="bi bi-heart-fill me-1"></i> ЛапиХъб
                    </span>
                </a>

                <button class="navbar-toggler" type="button" data-bs-toggle="collapse" data-bs-target="#mainNav"
                        aria-controls="mainNav" aria-expanded="false" aria-label="Меню">
                    <span class="navbar-toggler-icon"></span>
                </button>

                <div class="collapse navbar-collapse" id="mainNav">
                    <ul class="navbar-nav me-auto mb-2 mb-lg-0">
                        <li class="nav-item">
                            <a class="nav-link" asp-controller="Home" asp-action="Index">Начало</a>
                        </li>

                        <li class="nav-item">
                            <a class="nav-link" asp-controller="Products" asp-action="Index">Продукти</a>
                        </li>

                        <li class="nav-item">
                            <a class="nav-link" asp-controller="Home" asp-action="Contacts">Контакти</a>
                        </li>

                        @if (isAuthenticated)
                        {
                            <li class="nav-item">
                                <a class="nav-link" asp-controller="Dashboard" asp-action="Index">Табло</a>
                            </li>
                        }

                        @if (isAdmin)
                        {
                            <li class="nav-item dropdown position-relative">
                                <a class="nav-link dropdown-toggle" href="#" id="adminDropdown" role="button"
                                   data-bs-toggle="dropdown" aria-expanded="false">
                                    Администриране
                                </a>
                                <ul class="dropdown-menu dropdown-menu-end" aria-labelledby="adminDropdown">
                                    <li>
                                        <a class="dropdown-item" asp-controller="Admin" asp-action="Index">
```

```
                        data-bs-toggle="dropdown" aria-expanded="false">
                        Администриране
                    </a>
                    <ul class="dropdown-menu dropdown-menu-end" aria-labelledby="adminDropdown">
                        <li>
                            <a class="dropdown-item" asp-controller="Admin" asp-action="Index">
                                Админ табло
                            </a>
                        </li>
                        <li><hr class="dropdown-divider"></li>
                        <li>
                            <a class="dropdown-item" asp-controller="AdminProducts" asp-action="Index">
                                Управление на продукти
                            </a>
                        </li>
                        <li>
                            <a class="dropdown-item" asp-controller="AdminCategories" asp-action="Index">
                                Управление на категории
                            </a>
                        </li>
                        <li>
                            <a class="dropdown-item" asp-controller="AdminOrders" asp-action="Index">
                                Управление на поръчки
                            </a>
                        </li>
                        <li>
                            <a class="dropdown-item" asp-controller="AdminUsers" asp-action="Index">
                                Управление на потребители
                            </a>
                        </li>
                    </ul>
                </li>
            }
        </ul>

        <ul class="navbar-nav ms-auto align-items-lg-center gap-lg-2">
            @if (isAuthenticated)
            {
                <li class="nav-item">
                    <span class="navbar-text hello-text">
                        <i class="bi bi-person-circle me-1"></i> Здравей, @firstName
                    </span>
                </li>

                <li class="nav-item">
                    <form class="d-inline" asp-area="Identity" asp-page="/Account/Logout" method="post">
                        @Html.AntiForgeryToken()
                        <button type="submit" class="btn btn-outline-dark btn-sm">
```

```
193                         <form class="d-inline" asp-area="Identity" asp-page="/Account/Logout" method="post">
194                             @Html.AntiForgeryToken()
195                             <button type="submit" class="btn btn-outline-dark btn-sm">
196                                 Изход
197                             </button>
198                         </form>
199                     </li>
200                 }
201                 else
202                 {
203                     <li class="nav-item">
204                         <a class="nav-link" asp-area="Identity" asp-page="/Account/Login">Вход</a>
205                     </li>
206                     <li class="nav-item">
207                         <a class="btn btn-lapihub btn-sm" asp-area="Identity" asp-page="/Account/Register">
208                             Регистрация
209                         </a>
210                     </li>
211                 }
212             </ul>
213         </div>
214     </div>
215 </nav>
216 </header>
217
218 <main class="flex-fill">
219     <div class="container py-4 pb-3">
220         @RenderBody()
221     </div>
222 </main>
223
224 <footer class="py-4 mt-auto">
225     <div class="container d-flex flex-column flex-md-row justify-content-between align-items-center gap-2">
226         <div>
227             <strong>ЛапиХъб</strong> – магазин за консумативи и радост за домашни любимци.
228         </div>
229         <div class="text-muted">
230             © @DateTime.Now.Year – Всички права запазени.
231         </div>
232     </div>
233 </footer>
234
235 <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js"
236         integrity="sha384-YvpcrYf0tY3lHB60NNkmXc5s9fDVZLESaAA55NDzOxhy9GkcIdslK1eN7N6jIeHz"
237         crossorigin="anonymous"></script>
238
239     @RenderSection("Scripts", required: false)
240 </body>
```

Приложение 13: DbSeeder.cs



```
public static class DbSeeder
{
    1 reference
    public static async Task SeedRolesAndAdminAsync(IServiceProvider serviceProvider)
    {
        var roleManager = serviceProvider.GetRequiredService<RoleManager<IdentityRole>>();
        var userManager = serviceProvider.GetRequiredService<UserManager<ApplicationUser>>();

        string[] roles = { "Admin", "User" };

        foreach (var role in roles)
        {
            var roleExists = await roleManager.RoleExistsAsync(role);
            if (!roleExists)
            {
                await roleManager.CreateAsync(new IdentityRole(role));
            }
        }

        var adminEmail = "admin@lapihub.bg";
        var adminPassword = "admin123";

        var adminUser = await userManager.FindByEmailAsync(adminEmail);

        if (adminUser == null)
        {
            adminUser = new ApplicationUser
            {
                UserName = adminEmail,
                Email = adminEmail,
                EmailConfirmed = true,
                PhoneNumber = "0888000000",
                PhoneNumberConfirmed = true,
                FirstName = "Админ",
                LastName = "ЛапиХъб"
            };

            var createResult = await userManager.CreateAsync(adminUser, adminPassword);

            if (!createResult.Succeeded)
            {
                var errors = string.Join(Environment.NewLine, createResult.Errors.Select(e => e.Description));
                throw new Exception($"Грешка при създаване на admin потребителя:{Environment.NewLine}{errors}");
            }
        }
```

Приложение 14: Program.cs



```
{
    0 references
    public static async Task Main(string[] args)
    {
        var builder = WebApplication.CreateBuilder(args);

        var connectionString = builder.Configuration.GetConnectionString("DefaultConnection")
            ?? throw new InvalidOperationException("♦♦♦♦♦♦ connection string 'DefaultConnection'.");

        builder.Services.AddDbContext<ApplicationDbContext>(options =>
            options.UseSqlServer(connectionString));

        builder.Services.AddDatabaseDeveloperPageExceptionFilter();

        builder.Services
.AddDefaultIdentity<ApplicationUser>(options =>
{
    options.SignIn.RequireConfirmedAccount = false;

    options.Password.RequiredLength = 4;
    options.Password.RequireDigit = false;
    options.Password.RequireLowercase = false;
    options.Password.RequireUppercase = false;
    options.Password.RequireNonAlphanumeric = false;
    options.Password.RequiredUniqueChars = 0;
})
.AddRoles<IdentityRole>()
.AddEntityFrameworkStores<ApplicationDbContext>();

        builder.Services.AddControllersWithViews();

        var app = builder.Build();

        using (var scope = app.Services.CreateScope())
        {
            var services = scope.ServiceProvider;
            await DbSeeder.SeedRolesAndAdminAsync(services);
        }

        if (app.Environment.IsDevelopment())
        {
            app.UseMigrationsEndPoint();
        }
        else
        {
            app.UseExceptionHandler("/Home/Error");
```

```
            var app = builder.Build();

            using (var scope = app.Services.CreateScope())
            {
                var services = scope.ServiceProvider;
                await DbSeeder.SeedRolesAndAdminAsync(services);
            }

            if (app.Environment.IsDevelopment())
            {
                app.UseMigrationsEndPoint();
            }
            else
            {
                app.UseExceptionHandler("/Home/Error");
                app.UseHsts();
            }

            app.UseHttpsRedirection();
            app.UseStaticFiles();

            app.UseRouting();

            app.UseAuthentication();
            app.UseAuthorization();

            app.MapControllerRoute(
                name: "default",
                pattern: "{controller=Home}/{action=Index}/{id?}");

            app.MapRazorPages();

            app.Run();
        }
    }
}
```

Приложение 15: Home/Index.cshtml (слайдър)

```
<div class="container px-0">
    <div id="homeCarousel" class="carousel slide hero-carousel mb-5" data-bs-ride="carousel">
        <div class="carousel-indicators">
            <button type="button" data-bs-target="#homeCarousel" data-bs-slide-to="0" class="active" aria-current="true" aria-label="Слайд 1"></button>
            <button type="button" data-bs-target="#homeCarousel" data-bs-slide-to="1" aria-label="Слайд 2"></button>
            <button type="button" data-bs-target="#homeCarousel" data-bs-slide-to="2" aria-label="Слайд 3"></button>
        </div>

        <div class="carousel-inner rounded-4 overflow-hidden shadow">
            <div class="carousel-item active">
                <img src="https://images.unsplash.com/photo-1517849845537-4d257902454a?auto=format&fit=crop&w=1400&q=80" class="d-block w-100" alt="Щастливо куче">
                <div class="carousel-caption text-start d-flex justify-content-start align-items-end h-100 pb-5">
                    <div class="carousel-caption-custom">
                        <h1 class="display-6 fw-bold">Всичко за щастливи лапички</h1>
                        <p class="mb-3">Открий качествена храна, играчки, лакомства и аксесоари за твоите любимци.</p>
                        <a href="/Products" class="btn btn-lapihub">Разгледай продуктите</a>
                    </div>
                </div>
            </div>

            <div class="carousel-item">
                <img src="https://images.unsplash.com/photo-1519052537078-e6302a4968d4?auto=format&fit=crop&w=1400&q=80" class="d-block w-100" alt="Котка">
                <div class="carousel-caption text-start d-flex justify-content-start align-items-end h-100 pb-5">
                    <div class="carousel-caption-custom">
                        <h2 class="fw-bold">Подбрани марки и грижа с характер</h2>
                        <p class="mb-3">В ЛапиХъб събираме любими продукти за кучета и котки на едно място.</p>
                        <a href="/Products" class="btn btn-lapihub">Към каталога</a>
                    </div>
                </div>
            </div>

            <div class="carousel-item">
                <img src="https://images.unsplash.com/photo-1548199973-03cce0bbc87b?auto=format&fit=crop&w=1400&q=80" class="d-block w-100" alt="Домашен любимец">
                <div class="carousel-caption text-start d-flex justify-content-start align-items-end h-100 pb-5">
                    <div class="carousel-caption-custom">
                        <h2 class="fw-bold">Уют, игра и грижа всеки ден</h2>
                        <p class="mb-3">Избери продукти, които правят ежедневието на любимеца ти по-забавно и по-комфортно.</p>
                        <a href="/Products" class="btn btn-lapihub">Пазарувай сега</a>
                    </div>
                </div>
            </div>
```

Приложение 15: ApplicatiomDbContext.cs

```
using Lapihub.Models;
using Microsoft.AspNetCore.Identity.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore;
using static System.Net.WebRequestMethods;

namespace Lapihub.Data
{
    23 references
    public class ApplicationDbContext : IdentityDbContext<ApplicationUser>
    {
        0 references
        public ApplicationDbContext(DbContextOptions<ApplicationDbContext> options)
            : base(options)
        {
        }

        15 references
        public DbSet<Category> Categories { get; set; } = null!;
        15 references
        public DbSet<Product> Products { get; set; } = null!;
        11 references
        public DbSet<CartItem> CartItems { get; set; } = null!;
        9 references
        public DbSet<Order> Orders { get; set; } = null!;
        1 reference
        public DbSet<OrderItem> OrderItems { get; set; } = null!;

        0 references
        protected override void OnModelCreating(ModelBuilder builder)
        {
            base.OnModelCreating(builder);

            builder.Entity<Product>()
                .Property(p => p.Price)
                .HasColumnType("decimal(18,2)");

            builder.Entity<Order>()
                .Property(o => o.TotalAmount)
                .HasColumnType("decimal(18,2)");

            builder.Entity<OrderItem>()
                .Property(oi => oi.Price)
                .HasColumnType("decimal(18,2)");

            builder.Entity<Product>()
                .HasOne(p => p.Category)
                .WithMany(c => c.Products)
```

Приложение 16: UML Диаграма

